

Dyrepermen — teknisk plan

Prosjekt: Webapplikasjon for oppfølging av husstandens kjæledyr **Stack:** ASP.NET Core MVC (.NET 9) · Entity Framework Core · PostgreSQL **Dokumentversjon:** 2.3 — august 2026 **Formål med dokumentet:** Fullstendig teknisk grunnlag som kan mates inn i et agentisk kodeverktøy (Cowork / Claude Code) for videre implementering.

1. Bakgrunn og mål

Familien har hund og katt. Eksisterende apper på markedet dekker delvis behovet, men ingen dekker alt:

Behov	DyreID	11pets	Egen løsning
Chipnummer og NKK-regnummer	Ja	Nei	Ja
Vektlogg over tid	Ja	Ja	Ja
Vaksine, ormekur, flåttmiddel med varsel	Delvis	Ja	Ja
Medisinlogg med dosering	Nei	Ja	Ja
Veterinærbesøk med kostnad	Nei	Ja	Ja
Forsikringsoversikt	Nei	Nei	Ja
Brukerdefinert fôrplan	Nei	Nei	Ja
Fôringslogg mellom flere personer	Nei	Delvis	Ja
Handleliste med kobling til dyr	Nei	Delvis	Ja
Delt tilgang mellom to voksne	Kun betalt	Delvis	Ja

Målet er én applikasjon der to brukere i samme husstand har full, delt tilgang til alle data om alle dyrene, med automatiske påminnelser om det som forfaller.

MVP — hva som skal bygges først

MVP er fase 1, 1b og 2 i kapittel 16. Ingenting annet. En fungerende MVP betyr:

- To brukere i samme husstand kan logge inn og forblir innlogget over en utrulling
- De ser de samme dyrene, og kan opprette, redigere og deaktivere dyr
- De kan registrere vekt og se historikken
- De kan registrere behandlinger med neste dato, og se hva som forfaller
- Dashbordet viser dyrekort, forfallende behandlinger og tomtilstander
- Appen kjører i Docker lokalt og er rullet ut på Render mot Neon

Alt annet — fôrplan, fôringslogg, medisiner, veterinærbesøk, forsikring, dokumenter, handleliste, dataeksport, kontosletting — er **utenfor MVP**. Skjemaet i kapittel 5 beskriver hele modellen fordi migrasjonene skal være riktige fra start, men tabellene uten tilhørende MVP-funksjon får ingen controller, ingen view og ingen tjeneste før sin fase.

Bygg ikke videre til neste fase før akseptansekriteriene for gjeldende fase er oppfylt og testene er grønne.

Ikke-mål (bevisst utelatt i versjon 1)

- Ingen GPS-sporing eller integrasjon mot chip-lesere
- Ingen integrasjon mot veterinærens journalsystem
- Ingen mobilapp — responsiv web holder
- Ingen flerspråklighet — norsk bokmål i hele grensesnittet

2. Teknologivalg og begrunnelser

Lag	Valg	Begrunnelse
Runtime	.NET 9	LTS-nær, god Linux-containerstøtte, moderne minimal hosting
Web	ASP.NET Core MVC + Razor Views	Server-rendret, innebygd validering og modellbinding
ORM	EF Core 9 + Npgsql	Migrasjoner, globale spørringsfiltre, LINQ
Database	PostgreSQL 16	Kravsatt. Åpen, portabel, ingen leverandørlåsing
Autentisering	ASP.NET Core Identity	Ferdig brukerhåndtering, passordhashing, cookie-auth
Frontend-interaktivitet	Bootstrap 5 + htmlx	Dekker behovet uten byggesteg
Filer	Lokalt volum i v1, objektlagring i v2	Enkelt å starte med, lett å bytte bak et interface
Hosting	Render (app) + Neon (database)	Se kapittel 13
Repo	Monorepo, ett Git-repositorium	Atomiske endringer på tvers av lagene, se kapittel 3

2.1 Om React — anbefaling: ikke i versjon 1

Applikasjonen er skjemadrevet CRUD. Det tyngste grensesnittelementet er en vektgraf og en avkrysningsliste. Dette er akkurat den typen applikasjon Razor Views løser best:

- Modellbinding og servervalidering fungerer uten duplisert logikk i to lag
- Ingen separat byggekjede, ingen node_modules, ingen CORS-oppsett
- Ett deploy-artefakt i stedet for to
- `asp-for`-tag helpers gir typesikre skjemaer koblet direkte mot ViewModel

htmx dekker det lille som trengs av dynamikk (legge til rad uten full sidelast, markere handlelistepunkt som kjøpt, laste inn vektgraf) med attributter i markup, uten et eget rammeverk.

Når React likevel blir riktig: hvis appen senere skal være en installerbar PWA med offline-støtte — for eksempel registrere føring uten dekning på hyttetur. Da splittes løsningen slik:

```
Dyrepermen.Web      → beholdes som admin-/desktopgrensesnitt
Dyrepermen.Api      → nytt prosjekt, eksponerer REST over samme Application-lag
dyrepermen-client/  → React + Vite, konsumerer API-et
```

Fordi all forretningslogikk allerede ligger i `Dyrepermen.Application`, koster denne splitten kun et nytt tynt API-lag.

Kontrollerne må derfor holdes tynne fra dag én — de skal kun mappe mellom ViewModel og tjeneste, aldri inneholde forretningsregler.

3. Monorepo og løsningsstruktur

3.1 Prinsipp

Alt som hører til produktet ligger i **ett Git-repositorium**: applikasjonskode, tester, infrastrukturfiler, CI-arbeidsflyter, dokumentasjon og verktøyskript. Ingen del av systemet krever at man klonet noe annet for å bygge, kjøre eller rulle ut.

Begrunnelsen for dette prosjektet spesifikt:

- **Atomiske endringer.** En skjemaendring berører entitet, migrasjon, tjeneste, view og test. I ett repo er det én commit som enten går gjennom eller ikke. Fordelt på flere repoer blir det fire pull requests som må landes i riktig rekkefølge.
- **Én sannhet om versjoner.** Pakkeversjoner defineres ett sted og gjelder alle prosjektene.
- **Fremtidig React-klient.** Splittes grensesnittet senere (kapittel 2.1), kommer klienten inn som en mappe i samme repo og deler typedefinisjoner og CI med resten.
- **Agentisk utvikling.** Et kodeverktøy som ser hele systemet i én trestruktur trenger ikke kontekst utenfra for å forstå

sammenhengene.

3.2 Trestruktur

```
dyrepermen/
├── .github/
│   └── workflows/
│       ├── bygg.yml           bygg og test ved push
│       ├── rull-ut.yml        utrulling til Render
│       ├── paminnelser.yml     daglig cron mot jobb-endepunkt
│       └── sikkerhetskopi.yml  ukentlig pg_dump
├── src/
│   ├── Dyrepermen.Domain/      klassebibliotek, ingen avhengigheter
│   │   ├── Entities/
│   │   ├── Enums/
│   │   └── Abstractions/       IHusstandsbundet m.m.
│   ├── Dyrepermen.Application/ forretningslogikk
│   │   ├── Interfaces/
│   │   ├── Services/
│   │   └── Dtos/
│   ├── Dyrepermen.Infrastructure/ EF Core, e-post, fillagring
│   │   ├── Persistence/
│   │   │   ├── DyrepermenDbContext.cs
│   │   │   ├── Configurations/
│   │   │   └── Migrations/
│   │   ├── Email/
│   │   └── Storage/
│   ├── Dyrepermen.Web/         MVC
│   │   ├── Controllers/
│   │   ├── Views/
│   │   ├── ViewModels/
│   │   ├── BackgroundServices/
│   │   └── wwwroot/
├── clients/                    reservert for fremtidig React-klient
│   └── .gitkeep
├── tests/
│   ├── Dyrepermen.Application.Tests/
│   └── Dyrepermen.Integration.Tests/
├── infra/
│   ├── Dockerfile
│   ├── compose.yaml           lokal Postgres, valgfritt hele stakken
│   ├── .env.example           mal for lokale hemmeligheter
│   └── render.yaml            tjenestedefinisjon
├── docs/
│   ├── plan.md                dette dokumentet
│   └── beslutninger/          ADR-er, én fil per beslutning
├── tools/
│   └── migrer.sh               genererer og kjører idempotent skript
├── .editorconfig
├── .gitignore                 se kapittel 3.6
├── .dockerignore              må ligge i rot, ikke i infra/
├── global.json
├── Directory.Build.props
├── Directory.Packages.props
├── Dyrepermen.sln
├── CLAUDE.md                  arbeidsinstruks for kodeagent
└── README.md
```

3.3 Filer som gjør monorepoet til mer enn en mappe

Uten disse fire er det bare prosjekter som ligger ved siden av hverandre.

`global.json` låser SDK-versjonen, slik at alle bygger med samme kompilator — lokalt, i CI og i containeren:

```
{
  "sdk": {
    "version": "9.0.100",
    "rollForward": "latestFeature"
  }
}
```

`Directory.Build.props` i rot gjelder alle prosjekter og fjerner duplisering i hver `.csproj`:

```
<Project>
  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <LangVersion>latest</LangVersion>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <WarningsNotAsErrors>CS1591</WarningsNotAsErrors>
    <InvariantGlobalization>false</InvariantGlobalization>
  </PropertyGroup>
</Project>
```

`InvariantGlobalization` må stå til `false`. Applikasjonen formaterer datoer og tall på norsk og konverterer tidspunkter til `Europe/Oslo` — med invariant globalisering finnes ikke tidssonedatabasen i containeren, og konverteringen kaster.

`Directory.Packages.props` aktiverer sentral pakkestyring. Versjoner står ett sted, `.csproj`-filene refererer bare pakkenavnet:

```
<Project>
  <PropertyGroup>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
    <CentralPackageTransitivePinningEnabled>true</CentralPackageTransitivePinningEnabled>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Npgsql.EntityFrameworkCore.PostgreSQL" Version="9.0.*" />
    <PackageVersion Include="EFCore.NamingConventions" Version="9.0.*" />
    <PackageVersion Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="9.0.*" />
    <PackageVersion Include="Microsoft.AspNetCore.DataProtection.EntityFrameworkCore" Version="9.0.*" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.*" />
    <PackageVersion Include="xunit" Version="2.9.*" />
    <PackageVersion Include="Testcontainers.PostgreSql" Version="4.*" />
  </ItemGroup>
</Project>
```

Dette er den viktigste enkeltfilen i et .NET-monorepo. Uten den ender fire prosjekter opp med tre ulike EF Core-versjoner, og feilen viser seg som uforståelige kjøretidsfeil i migrasjoner.

`.editorconfig` i rot gir samme formatering uansett editor. Sett minst `indent_size = 4` for `.cs`, `end_of_line = lf` for alt, og `charset = utf-8`.

3.4 Avhengighetsretning

Web → Application → Domain og Infrastructure → Application → Domain. Domain-laget refererer ingenting. Web refererer Infrastructure kun i `Program.cs` for DI-registrering.

Regelen håndheves ikke av kompilatoren alene — den må respekteres bevisst. Legger man en EF Core-avhengighet i Domain, er lagdelingen borte uten at noe feiler.

3.5 Konsekvenser for bygg og utrulling

Dockerfile ligger i `infra/`, men byggekonteksten er repo-roten. `COPY`-stier er relative til konteksten, ikke til Dockerfile-ens plassering:

```
docker build -f infra/Dockerfile -t dyrepermen .
```

Glemmer man `-f`-flagget eller setter konteksten til `infra/`, feiler bygget med at `.csproj`-filene ikke finnes. På Render settes dette som `dockerfilePath: infra/Dockerfile` og `dockerContext: .` i `render.yaml`.

`.dockerignore` **må ligge i repo-roten**, ikke i `infra/`. Docker leser den fra konteksten. Uten den kopieres `.git`, `bin`, `obj`, `docs` og `clients` inn i byggelaget og gjør bygget merkbart tregere.

CI-arbeidsflyter filtreres på sti, slik at en endring i `docs/` ikke utløser bygg og utrulling:

```
on:
  push:
    branches: [main]
    paths:
      - 'src/**'
      - 'tests/**'
      - 'infra/**'
      - 'Directory.*.props'
      - 'global.json'
```

Migrasjoner kjøres fra roten med eksplisitte prosjektsier, siden `dotnet ef` ellers gjetter feil i et repo med flere prosjekter:

```
dotnet ef migrations add <navn> \
  --project src/Dyrepermen.Infrastructure \
  --startup-project src/Dyrepermen.Web \
  --output-dir Persistence/Migrations
```

3.6 .gitignore

Ligger i repo-roten og dekker hele monorepoet. Delen under linjen «Prosjektspesifikt» er den viktige — resten er standard .NET-støy.

```
# --- Byggeartefakter ---
[Bb]in/
[Oo]bj/
[Oo]ut/
artifacts/
*.dll
*.pdb
*.exe

# --- Verktøy og editorer ---
.vs/
.vscode/*
!.vscode/extensions.json
!.vscode/launch.json
.idea/
*.user
*.suo
*.userosscache
*.sln.docstates
_ReSharper*/
.fake/

# --- Test og dekning ---
[Tt]est[Rr]esult*/
coverage*.json
coverage*.xml
```

```

*.trx
*.coverage
BenchmarkDotNet.Artifacts/

# --- Node, for fremtidig klient i clients/ ---
node_modules/
dist/
.vite/
*.tsbuildinfo
npm-debug.log*
pnpm-debug.log*

# --- Operativsystem ---
.DS_Store
Thumbs.db
desktop.ini

# =====
# Prosjektspesifikt – les denne delen
# =====

# Hemmeligheter. Skal ALDRI i repoet.
.env
infra/.env
*.env.local
appsettings.*.local.json
secrets.json
*.pfx
*.p12
*.key

# Eksempelfil skal derimot være med
!.env.example
!infra/.env.example

# Opplastede dokumenter (kapittel 13.1, fillager i v1)
opplastinger/
src/Dyrepermen.Web/wwwroot/opplastinger/

# Generert migrasjonsskript (kapittel 14.3)
migrations.sql

# Databasedumper fra sikkerhetskopi-jobben (kapittel 15)
*.sql.gz
*.dump
sikkerhetskopi/

# Lokale Docker-volumer om noen binder dem til disk
.docker-data/

```

Fire feller verdt å kjenne

.env må ligge både med og uten sti. Compose leser `infra/.env`, men det er lett å opprette en `.env` i roten under feilsøking. Begge må dekkles, ellers ligger databasepassordet i en commit.

!.env.example må stå etter **.env-regelen**. Git leser mønstrene i rekkefølge, og siste treff vinner. Står unntaket først, blir eksempelfilen ignorert likevel — og da vet ingen hvilke variabler som trengs.

Ikke ignorer Persistence/Migrations/. Migrasjonsfiler er kildekode og skal versjonshåndteres. Flere .NET-maler har `Migrations/` i sin ignorer-liste fra tider da man genererte dem på nytt hver gang. Gjør du det her, kan ingen andre bygge databasen, og utrulling har ingenting å kjøre.

Opplastingsmappen må ignoreres før første opplasting. Legges en vaksinasjonsattest inn før regelen finnes, ligger

den i historikken permanent — og `.gitignore` fjerner ikke noe som allerede er sporet.

3.7 `.dockerignore`

Ligger i repo-roten, ikke i `infra/`, fordi Docker leser den fra byggekonteksten (kapittel 3.5).

```
**/bin/
**/obj/
**/node_modules/
.git/
.github/
.vs/
.vscode/
.idea/
docs/
clients/
tests/
infra/compose.yaml
**/.env
*.md
!README.md
migrations.sql
*.sql.gz
```

`tests/` og `docs/` utelates bevisst — de trengs ikke i produksjonsbildet og er en betydelig andel av filene. `.git/` er som regel den største enkeltposten; uten den regelen kopieres hele historikken inn i byggelaget.

Merk at `clients/` er utelatt her. Bygges React-klienten senere som en del av samme container, må regelen fjernes og et eget byggesteg legges til i Dockerfile-en.

4. Domenemodell

4.1 Aggregater

Aggregat	Rot	Barn
Husstand	Husstand	Bruker , Handleliste
Dyr	Dyr	Vekt , Behandling , Medisin , Vetbesok , Forsikring , Dokument
Medisin	Medisin	Dose

`Husstand` er tenant-roten. Alle spørringer filtreres på `HusstandId`.

4.2 Entiteter

Husstand

Felt	Type	Merknad
Id	int	PK, identity
Navn	varchar(80)	f.eks. "Hjemme"
OpprettetDato	date	

Bruker — utvider `IdentityUser<int>`

Felt	Type	Merknad
Id	int	PK, fra Identity
HusstandId	int	FK, nullable inntil brukeren er tilknyttet
Visningsnavn	varchar(60)	

Dyr

Felt	Type	Merknad
Id	int	PK
HusstandId	int	FK, påkrevd
Navn	varchar(60)	
Art	char(1)	H = hund, K = katt
Rase	varchar(80)	
Kjonn	char(1)	T = tisper/hunn, H = hann
Fodselsdato	date	
ChipNr	varchar(15)	Globalt unikt. Norske chip starter på 578
RegNrNkk	varchar(20)	Globalt unikt, nullable
Kastrert	boolean	
BildeFilnavn	varchar(120)	nullable
ForingsloggAktiv	boolean	funksjonsbryter, arves fra husstandens standard
ForplanAktiv	boolean	funksjonsbryter, arves fra husstandens standard
Aktiv	boolean	Settes false i stedet for sletting

Vekt

Felt	Type	Merknad
Id	int	PK
DyrId	int	FK
VektGram	int	Heltall — unngår desimalavrundning
Dato	date	
RegistrertAvBrukerId	int	FK

Behandling — vaksine, ormekur, flåttmiddel, kloklipp, tannrens

Felt	Type	Merknad
Id	int	PK
DyrId	int	FK
Type	char(1)	V, O, F, K, T
Preparat	varchar(80)	f.eks. “Milbemax”
Dato	date	
NesteDato	date	nullable, driver påminnelser
Notat	varchar(500)	nullable

Medisin

Felt	Type	Merknad
Id	int	PK
DyrId	int	FK
Navn	varchar(80)	
Dose	varchar(40)	fritekst, f.eks. "1/2 tablett"
IntervallTimer	int	0 = ved behov
StartDato	date	
SluttDato	date	nullable, null = pågående

Dose — logg over faktisk gitt medisin

Felt	Type	Merknad
Id	int	PK
MedisinId	int	FK
GittTid	timestampz	
GittAvBrukerId	int	FK — hindrer dobbeltdosering

Forplan — brukerdefinert fôrregel per dyr

Felt	Type	Merknad
Id	int	PK
DyrId	int	FK
Metode	char(1)	P = prosent av kroppsvekt, G = fast gram
ProsentTidels	int	nullable. 50 = 5,0 %. Kun ved metode P
GramPerDag	int	nullable. Kun ved metode G
AntallMaltider	int	standard 2
Fornavn	varchar(80)	navn på fôret, nullable
Notat	varchar(300)	nullable
Aktiv	boolean	kun én aktiv plan per dyr
OpprettetDato	date	

Foring — logg over gitte måltider

Felt	Type	Merknad
Id	int	PK
DyrId	int	FK
Tidspunkt	timestampz	settes automatisk ved registrering
MengdeGram	int	nullable — kan hukes av uten mengde
GittAvBrukerId	int	FK
Kommentar	varchar(200)	nullable

HusstandInvitasjon — forhåndsgodkjent e-postadresse

Felt	Type	Merknad
Id	int	PK
HusstandId	int	FK
Epost	varchar(256)	normalisert til små bokstaver
InnlostAvBrukerId	int	nullable. Satt når adressen registrerte seg
InnlostTid	timestamp tz	nullable
OpprettetAvBrukerId	int	nullable, FK
OpprettetDato	date	

HusstandInnstilling — standardverdier for nye dyr, ikke overstyring

Felt	Type	Merknad
HusstandId	int	PK og FK
ForingsloggStandard	boolean	standard false. Kopieres til nye dyr
ForplanStandard	boolean	standard true. Kopieres til nye dyr
VarslerAktiv	boolean	standard true. Gjelder e-postutsending, er husstands nivå

Vetbesok

Felt	Type	Merknad
Id	int	PK
DyrId	int	FK
Dato	date	
Klinikk	varchar(100)	
Arsak	varchar(200)	
Diagnose	varchar(200)	nullable
KostnadKr	int	hele kroner
ForsikringKrevd	boolean	

Forsikring

Felt	Type	Merknad
Id	int	PK
DyrId	int	FK
Selskap	varchar(80)	
PoliseNr	varchar(40)	
ArspremieKr	int	
Egenandel	int	
FornyesDato	date	driver påminnelse

Dokument

Felt	Type	Merknad
Id	int	PK
DyrlId	int	FK
Filnavn	varchar(200)	lagret navn (GUID + ext)
Originalnavn	varchar(200)	brukerens filnavn
Kategori	char(1)	V = vaksinebok, J = journal, K = kvittering, A = annet
OpplastetDato	date	

Handleliste

Felt	Type	Merknad
Id	int	PK
HusstandId	int	FK — ikke DyrlId, dette er husstandsoppgaver
DyrlId	int	FK, nullable — valgfri kobling
Tekst	varchar(120)	
Antall	int	
Status	char(1)	A = aktiv, K = kjøpt
OpprettetAvBrukerId	int	FK
OpprettetDato	date	

5. Databaseskjema (PostgreSQL)

EF Core genererer migrasjonene, men skjemaet under er fasiten som migrasjonene skal produsere. Alle tabellnavn i entall, snake_case kolonner via Npgsql-konvensjon.

5.1 Tabeller EF Core lager selv

DDL-en under dekker domenetabellene. I tillegg oppretter migrasjonene:

- **ASP.NET Core Identity:** `asp_net_users`, `asp_net_roles`, `asp_net_user_claims`, `asp_net_user_logins`, `asp_net_user_tokens`, `asp_net_user_roles`, `asp_net_role_claims`. `asp_net_users` utvides med kolonnene `husstand_id` `INT REFERENCES husstand(id)` og `visningsnavn` `VARCHAR(60)` via `Bruker : IdentityUser<int>`.
- **Data Protection:** `data_protection_keys`, fra `PersistKeysToDbContext` (kapittel 11.2). Denne tabellen må aldri tømmes — det logger ut alle brukere.

Alle `*_av_bruker_id`-kolonner i domenetabellene refererer `asp_net_users(id)`. Fremmednøkklene er utelatt fra DDL-en under fordi Identity-tabellen opprettes i samme migrasjon; de konfigureres i entitetskonfigurasjonene.

5.2 Domenetabeller

```
CREATE TABLE husstand (
    id            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    navn          VARCHAR(80) NOT NULL,
    opprettet_dato DATE      NOT NULL DEFAULT CURRENT_DATE
);

CREATE TABLE dyr (
    id            INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    husstand_id  INT          NOT NULL REFERENCES husstand(id),
    navn         VARCHAR(60) NOT NULL,
    art          CHAR(1)     NOT NULL,
    rase         VARCHAR(80),
```

```

    kjonn          CHAR(1)          NOT NULL,
    fodselsdato    DATE,
    chip_nr        VARCHAR(15),
    reg_nr_nkk     VARCHAR(20),
    kastrert       BOOLEAN          NOT NULL DEFAULT FALSE,
    bilde_filnavn  VARCHAR(120),
    foringslogg_aktiv BOOLEAN      NOT NULL DEFAULT FALSE,
    forplan_aktiv  BOOLEAN          NOT NULL DEFAULT TRUE,
    aktiv          BOOLEAN          NOT NULL DEFAULT TRUE,
    CONSTRAINT ck_dyr_art CHECK (art IN ('H','K')),
    CONSTRAINT ck_dyr_kjonn CHECK (kjonn IN ('T','H')),
    CONSTRAINT ck_dyr_chip_lengde
        CHECK (chip_nr IS NULL OR char_length(chip_nr) = 15)
);
CREATE INDEX ix_dyr_husstand ON dyr(husstand_id);

-- Unikhet er global, ikke per husstand. Chipnummer er unike på verdensbasis,
-- og NKK-regnummer er unike i registeret.
CREATE UNIQUE INDEX ux_dyr_chip
    ON dyr(chip_nr) WHERE chip_nr IS NOT NULL;
CREATE UNIQUE INDEX ux_dyr_regnr
    ON dyr(upper(reg_nr_nkk)) WHERE reg_nr_nkk IS NOT NULL;

CREATE TABLE vekt (
    id                INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id            INT NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    vekt_gram         INT NOT NULL CHECK (vekt_gram > 0),
    dato              DATE NOT NULL,
    registrert_av Bruker_id INT
);
CREATE INDEX ix_vekt_dyr_dato ON vekt(dyr_id, dato DESC);

CREATE TABLE behandling (
    id                INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id            INT NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    type              CHAR(1) NOT NULL,
    preparat          VARCHAR(80),
    dato              DATE NOT NULL,
    neste_dato        DATE,
    notat             VARCHAR(500),
    CONSTRAINT ck_behandling_type CHECK (type IN ('V','O','F','K','T'))
);
CREATE INDEX ix_behandling_neste ON behandling(neste_dato)
    WHERE neste_dato IS NOT NULL;

CREATE TABLE medisin (
    id                INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id            INT NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    navn              VARCHAR(80) NOT NULL,
    dose              VARCHAR(40) NOT NULL,
    intervall_timer   INT NOT NULL DEFAULT 0,
    start_dato        DATE NOT NULL,
    slutt_dato        DATE
);

CREATE TABLE dose (
    id                INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    medisin_id        INT NOT NULL REFERENCES medisin(id) ON DELETE CASCADE,
    gitt_tid           TIMESTAMPTZ NOT NULL,
    gitt_av Bruker_id INT
);

```

```

CREATE INDEX ix_dose_medisin_tid ON dose(medisin_id, gitt_tid DESC);

CREATE TABLE forplan (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id INT NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    metode CHAR(1) NOT NULL,
    prosent_tidels INT,
    gram_per_dag INT,
    antall_maltider INT NOT NULL DEFAULT 2 CHECK (antall_maltider BETWEEN 1 AND 6),
    fornavn VARCHAR(80),
    notat VARCHAR(300),
    aktiv BOOLEAN NOT NULL DEFAULT TRUE,
    opprettet_dato DATE NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT ck_forplan_metode CHECK (metode IN ('P', 'G')),
    CONSTRAINT ck_forplan_verdi CHECK (
        (metode = 'P' AND prosent_tidels IS NOT NULL
         AND prosent_tidels BETWEEN 1 AND 300
         AND gram_per_dag IS NULL)
        OR (metode = 'G' AND gram_per_dag IS NOT NULL
         AND gram_per_dag > 0
         AND prosent_tidels IS NULL))
);

CREATE UNIQUE INDEX ux_forplan_aktiv ON forplan(dyr_id) WHERE aktiv;

CREATE TABLE foring (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id INT NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    tidspunkt TIMESTAMPTZ NOT NULL DEFAULT now(),
    mengde_gram INT CHECK (mengde_gram IS NULL OR mengde_gram > 0),
    gitt_av Bruker_id INT,
    kommentar VARCHAR(200)
);

CREATE INDEX ix_foring_dyr_tid ON foring(dyr_id, tidspunkt DESC);

CREATE TABLE husstand_invtasjon (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    husstand_id INT NOT NULL REFERENCES husstand(id) ON DELETE CASCADE,
    epost VARCHAR(256) NOT NULL,
    innlost_av Bruker_id INT,
    innlost_tid TIMESTAMPTZ,
    opprettet_av Bruker_id INT,
    opprettet_dato DATE NOT NULL DEFAULT CURRENT_DATE
);

-- Én ventende invitasjon per adresse på tvers av hele systemet.
CREATE UNIQUE INDEX ux_invtasjon_epost
    ON husstand_invtasjon(lower(epost)) WHERE innlost_tid IS NULL;

-- Standardverdier som kopieres til nye dyr ved opprettelse.
-- Overstyrer aldri en bryter som allerede står på et dyr.
CREATE TABLE husstand_innstilling (
    husstand_id INT PRIMARY KEY REFERENCES husstand(id) ON DELETE CASCADE,
    foringslogg_standard BOOLEAN NOT NULL DEFAULT FALSE,
    forplan_standard BOOLEAN NOT NULL DEFAULT TRUE,
    varsler_aktiv BOOLEAN NOT NULL DEFAULT TRUE
);

CREATE TABLE vetbesok (
    id INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id INT NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    dato DATE NOT NULL,
    klinikk VARCHAR(100),

```

```

    arsak          VARCHAR(200) NOT NULL,
    diagnose       VARCHAR(200),
    kostnad_kr     INT          NOT NULL DEFAULT 0,
    forsikring_krevd BOOLEAN    NOT NULL DEFAULT FALSE
);

CREATE TABLE forsikring (
    id              INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id         INT          NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    selskap        VARCHAR(80) NOT NULL,
    polise_nr      VARCHAR(40) NOT NULL,
    arspremie_kr   INT          NOT NULL DEFAULT 0,
    egenandel      INT          NOT NULL DEFAULT 0,
    fornyes_dato   DATE
);

CREATE TABLE dokument (
    id              INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    dyr_id         INT          NOT NULL REFERENCES dyr(id) ON DELETE CASCADE,
    filnavn        VARCHAR(200) NOT NULL,
    originalnavn   VARCHAR(200) NOT NULL,
    kategori       CHAR(1)      NOT NULL,
    opplastet_dato DATE          NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT ck_dokument_kategori CHECK (kategori IN ('V','J','K','A'))
);

CREATE TABLE handleliste (
    id              INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    husstand_id    INT          NOT NULL REFERENCES husstand(id),
    dyr_id         INT          REFERENCES dyr(id),
    tekst          VARCHAR(120) NOT NULL,
    antall         INT          NOT NULL DEFAULT 1,
    status         CHAR(1)      NOT NULL DEFAULT 'A',
    opprettet_av Bruker_id INT,
    opprettet_dato DATE          NOT NULL DEFAULT CURRENT_DATE,
    CONSTRAINT ck_handleliste_status CHECK (status IN ('A','K'))
);

CREATE INDEX ix_handleliste_aktiv ON handleliste(husstand_id)
WHERE status = 'A';

```

Merknader til skjemavalgene

- `CHAR(1)` -koder med `CHECK` -constraint i stedet for Postgres-enums. Enums krever egen migrasjon ved endring; en `CHECK` er én `ALTER TABLE`.
- `vekt_gram` som `INT` — 27,4 kg lagres som 27400. All aritmetikk blir eksakt, formatering skjer i visningslaget.
- Partielle indekser på `neste_dato` og aktiv handleliste — påminnelsesjobben og forsiden er de eneste hyppige spørringene, og begge treffer kun en delmengde av radene.
- Ingen `ON DELETE CASCADE` fra `husstand` — sletting av `husstand` skal aldri skje utilsiktet.
- Dyr slettes ikke, de deaktiveres (`aktiv = false`). Historikk om et dyr som er gått bort skal bevares.
- `CHAR(15)` ble forkastet for `chip_nr`. PostgreSQL blank-padder `char(n)`, slik at verdien kommer tilbake fra EF Core med etterfølgende mellomrom og ødelegger både sammenligning og unikhetssjekk. `VARCHAR(15)` med lengde-CHECK gir samme garanti uten padding.
- `prosent_tidels` lagres i tidelsprosent ($50 = 5,0\%$) slik at hele modellen holder seg til `INT`. Øvre grense 300 (30 %) er en ren tastefeilsperre, ikke en faglig anbefaling.
- `CHECK`-en på `forplan` gjør de to metodene gjensidig utelukkende. Databasen skal ikke kunne inneholde en plan som er halvt prosentbasert og halvt fast.

5.3 Unikhet på chipnummer og regnummer

Kravet er at samme dyr aldri kan registreres to ganger, mens flere dyr av samme art og rase skal være uproblematisk. Tre ting må være på plass for at de partielle unike indeksene faktisk skal virke:

Tom streng må normaliseres til NULL før lagring. Et skjemafelt som ikke fylles ut sender "", ikke null. To dyr uten chipnummer vil da kollidere, siden '' er en verdi mens NULL ikke deltar i unikhetssjekken.

```
public static string? TomTilNull(this string? s)
    => string.IsNullOrEmpty(s) ? null : s.Trim();
```

Kollisjon kan komme fra en rad du ikke ser. Det globale query-filteret skjuler deaktiverte dyr, men indeksen ser dem. Registrering av et dyr med chipnummer som allerede tilhører et deaktivert dyr gir `PostgresException` med `SQLSTATE 23505`. Fang den og oversett til en forståelig melding:

```
catch (DbUpdateException ex)
    when (ex.InnerException is PostgresException { SqlState: "23505" } pg)
{
    var felt = pg.ConstraintName switch
    {
        "ux_dyr_chip" => "Chipnummeret er allerede registrert på et dyr.",
        "ux_dyr_regrnr" => "Registreringsnummeret er allerede i bruk.",
        _ => "Verdien finnes allerede."
    };
    ModelState.AddModelError(string.Empty, felt);
}
```

Unikheten er global på tvers av husstander. Det er tilsiktet. Konsekvensen er at feilmeldingen ikke må avsløre hvilken husstand dyret tilhører — hold teksten nøytral, slik som over.

5.4 Samtidighet

To brukere som redigerer samme rad samtidig gir tapt oppdatering uten videre. Npgsql kan bruke Postgres' interne `xmin`-kolonne som concurrency token, uten egen kolonne i skjemaet:

```
b.Entity<Dyr>().UseXminAsConcurrencyToken();
b.Entity<Forplan>().UseXminAsConcurrencyToken();
```

`DbUpdateConcurrencyException` fanges i kontrolleren og vises som «Noen andre endret dette mens du redigerte — last siden på nytt.» Billig å legge inn nå, vondt å ettermontere.

6. Typer og kontrakter

Kodeeksemplene i dokumentet refererer navigasjonsegenskaper, hjelpemetoder og grensesnitt. De er samlet her så de ikke må utledes.

6.1 Navigasjonsegenskaper

```

public class Dyr
{
    public int Id { get; set; }
    public int HusstandId { get; set; }
    public Husstand Husstand { get; set; } = null!;

    public ICollection<Vekt> Vekter { get; set; } = new List<Vekt>();
    public ICollection<Behandling> Behandlinger { get; set; } = new List<Behandling>();
    public ICollection<Medisin> Medisiner { get; set; } = new List<Medisin>();
    public ICollection<Foring> Foringer { get; set; } = new List<Foring>();
    public ICollection<Vetbesok> Vetbesok { get; set; } = new List<Vetbesok>();
    public ICollection<Forsikring> Forsikringer { get; set; } = new List<Forsikring>();
    public ICollection<Dokument> Dokumenter { get; set; } = new List<Dokument>();
    public ICollection<Forplan> Forplaner { get; set; } = new List<Forplan>();
}

```

Barneentiteter har `Dyr Dyr { get; set; }` og `int DyrId`. Dose har `Medisin Medisin`. Foring, Vekt, Dose og Handleliste har en nullbar `Bruker? GittAv / RegistrertAv / OpprettetAv` — nullbar fordi brukeren kan være slettet (kapittel 12.5).

6.2 Markørgrensesnitt

```

public interface IHusstandsbundet { }

```

Implementeres av alle entiteter som skal ha query-filter: `Dyr`, `Vekt`, `Behandling`, `Medisin`, `Dose`, `Foring`, `Forplan`, `Vetbesok`, `Forsikring`, `Dokument`, `Handleliste`, `HusstandInnstilling`, `HusstandInvitasjon`. Testen i kapittel 17 feiler hvis en implementasjon mangler filter.

6.3 Hjelpemetoder

```

public static class ClaimsPrincipalExtensions
{
    public static int? BrukerId(this ClaimsPrincipal bruker)
    {
        var verdi = bruker.FindFirstValue(ClaimTypes.NameIdentifier);
        return int.TryParse(verdi, out var id) ? id : null;
    }
}

public static class StringExtensions
{
    public static string? TomTilNull(this string? s)
        => string.IsNullOrEmpty(s) ? null : s.Trim();
}

```

6.4 Enums

```

public enum Formetode { Prosent, Gram }
public enum Art { Hund, Katt }
public enum Kjonn { Tispe, Hann }
public enum BehandlingType { Vaksine, Ormekur, Flatt, Klokklipp, Tannrens }
public enum DokumentKategori { Vaksinebok, Journal, Kvittering, Annet }
public enum HandlelisteStatus { Aktiv, Kjopt }

```

Alle lagres som `char(1)` med eksplisitt `HasConversion` — se kapittel 8.1. Bokstavkodene er `P/G`, `H/K`, `T/H`, `V/O/F/K/T`, `V/J/K/A`, `A/K`.

6.5 Resultattyper

```

public enum LeggTilResultat
{ LagtTil, VenterPaRegistrering, AlleredeMedlem, TilhorerAnnenHusstand }

public enum SlettResultat
{ Ok, FeilPassord, MaBekreftHusstandsletting }

public sealed record ForplanResultat(
    bool HarPlan, bool ManglerVekt,
    int GramPerDag, int AntallMaltider,
    int? GrunnlagVektGram, DateOnly? GrunnlagDato)
{
    public static ForplanResultat IngenPlan() => new(false, false, 0, 0, null, null);
    public static ForplanResultat ManglerVektgrunnlag() => new(true, true, 0, 0, null, null);
    public static ForplanResultat Ok(int gram, int maltider,
        int? grunnlagVektGram = null, DateOnly? grunnlagDato = null)
        => new(true, false, gram, maltider, grunnlagVektGram, grunnlagDato);
}

public sealed record Paminnelse(
    string DyreNavn, Kilde Kilde, string Tekst, DateOnly Dato);

public enum Kilde { Behandling, Medisin, Forsikring }

```

6.6 Tjenestegrensesnitt

Alle registreres Scoped i `Dyrepermen.Application`:

Grensesnitt	Ansvar
<code>IHusstandContext</code>	Gjeldende husstand, se kapittel 7.2
<code>IDyrService</code>	CRUD på dyr, normalisering av chip og regnr, funksjonsbrytere
<code>IForplanService</code>	<code>BeregnAktiv</code> , opprett og deaktiver plan
<code>IForingService</code>	<code>Registrer</code> , <code>RedigerTid</code> , <code>SistMatet</code>
<code>IMedisinService</code>	<code>LoggDose</code> med dobbeltdoseringssjekk
<code>IPaminnelseService</code>	<code>ForfallerInnen(dager)</code>
<code>IHusstandService</code>	Opprett husstand, <code>LeggTilMedlem</code> , fjern medlem, forlat
<code>IKontoService</code>	Endre profil, eksporter data, <code>SlettBruker</code>
<code>IVarselSender</code>	<code>SendDagligOppsummering</code> — e-post
<code>IFillager</code>	Lagre, hent og slett dokumenter

Kodeeksemplene i dokumentet viser implementasjonene direkte for lesbarhet. I koden skal de ligge bak grensesnittene over, slik at controllerne kan enhetstestes.

7. Dataadgang og multi-tenancy

7.1 DbContext

```

public class DyrepermenDbContext : IdentityDbContext<Bruker, IdentityRole<int>, int>
{
    private readonly IHusstandContext _husstand;

    public DyrepermenDbContext(
        DbContextOptions<DyrepermenDbContext> options,
        IHusstandContext husstand) : base(options)
    {

```

```

        _husstand = husstand;
    }

    public DbSet<Dyr> Dyr => Set<Dyr>();
    public DbSet<Vekt> Vekt => Set<Vekt>();
    public DbSet<Behandling> Behandling => Set<Behandling>();
    public DbSet<Medisin> Medisin => Set<Medisin>();
    public DbSet<Dose> Dose => Set<Dose>();
    public DbSet<Vetbesok> Vetbesok => Set<Vetbesok>();
    public DbSet<Forsikring> Forsikring => Set<Forsikring>();
    public DbSet<Dokument> Dokument => Set<Dokument>();
    public DbSet<Handleliste> Handleliste => Set<Handleliste>();

    protected override void OnModelCreating(ModelBuilder b)
    {
        base.OnModelCreating(b);
        b.ApplyConfigurationsFromAssembly(typeof(DyrepermenDbContext).Assembly);

        b.Entity<Dyr>()
            .HasQueryFilter(d => d.HusstandId == _husstand.HusstandId && d.Aktiv);

        b.Entity<Handleliste>()
            .HasQueryFilter(h => h.HusstandId == _husstand.HusstandId);

        b.Entity<Vekt>()
            .HasQueryFilter(v => v.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Behandling>()
            .HasQueryFilter(x => x.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Medisin>()
            .HasQueryFilter(m => m.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Dose>()
            .HasQueryFilter(d => d.Medisin.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Vetbesok>()
            .HasQueryFilter(x => x.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Forsikring>()
            .HasQueryFilter(f => f.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Dokument>()
            .HasQueryFilter(x => x.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Forplan>()
            .HasQueryFilter(f => f.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<Foring>()
            .HasQueryFilter(f => f.Dyr.HusstandId == _husstand.HusstandId);

        b.Entity<HusstandInnstilling>()
            .HasQueryFilter(i => i.HusstandId == _husstand.HusstandId);

        b.Entity<HusstandInvitasjon>()
            .HasQueryFilter(i => i.HusstandId == _husstand.HusstandId);
    }
}

```

7.2 Husstandskontekst

```

public interface IHusstandContext
{
    int HusstandId { get; }
}

public sealed class DbHusstandContext : IHusstandContext
{
    private readonly IHttpContextAccessor _accessor;
    private readonly DyrepermenDbContext _db;
    private int? _bufret;

    public DbHusstandContext(
        IHttpContextAccessor accessor, DyrepermenDbContext db)
    {
        _accessor = accessor;
        _db = db;
    }

    public int HusstandId
    {
        get
        {
            if (_bufret is not null) return _bufret.Value;

            var brukerId = _accessor.HttpContext?.User.BrukerId();
            if (brukerId is null) return (_bufret = 0).Value;

            _bufret = _db.Users
                .Where(u => u.Id == brukerId.Value)
                .Select(u => u.HusstandId ?? 0)
                .FirstOrDefault();

            return _bufret.Value;
        }
    }
}

```

Vedlikeholdsregel: hver nye husstandsbundne tabell skal ha et filter her. Listen over må ha like mange oppføringer som det finnes husstandsbundne `DbSet`-er. Integrasjonstesten i kapittel 17 skal telle dem og feile hvis en mangler — ellers oppdages hullet først når noen ser andres data.

Returnerer 0 for uautentiserte brukere, som gir tomt resultatsett — fail closed.

Hvorfor database og ikke claim. En claim-basert variant (`husstand_id` lagt på ved innlogging) er raskere, men blir foreldet: et medlem kan legges til i en husstand av noen andre, og serveren kan ikke oppdatere en annen brukers informasjonskapsel. Med 30 dagers vedvarende innlogging vil personen da se en tom applikasjon i ukevis. Se kapittel 12.3.1.

Tjenesten registreres `Scoped`, så oppslaget skjer én gang per forespørsel uansett hvor mange query-filtre som leser den.

Kritisk: bakgrunnsjobben kjører uten HTTP-kontekst. Den må bruke en egen `SystemHusstandContext` og eksplisitt `IgnoreQueryFilters()`, ellers ser den ingen data.

7.3 Lokalisering

Applikasjonen kjører med norsk kultur, ikke invariant. Uten dette tolkes `27,4` som ugyldig i vektskjemaet og datoer parses på amerikansk format:

```
var norsk = new CultureInfo("nb-NO");

builder.Services.Configure<RequestLocalizationOptions>(o =>
{
    o.DefaultRequestCulture = new RequestCulture(norsk, norsk);
    o.SupportedCultures      = new[] { norsk };
    o.SupportedUICultures    = new[] { norsk };
    o.RequestCultureProviders.Clear();
});

// I pipeline, før UseRouting:
app.UseRequestLocalization();
```

`RequestCultureProviders.Clear()` er viktig. Uten den leser ASP.NET Core `Accept-Language` fra nettleseren, og en bruker med engelsk nettleser får punktum som desimalskilletegn mens skjemaet forventer komma. Applikasjonen er ensidig norsk — da skal kulturen være fast, ikke forhandlet.

Tidssone: all lagring skjer i UTC (`timestampz`). Konvertering til visning gjøres ett sted:

```
private static readonly TimeZoneInfo Oslo =
    TimeZoneInfo.FindSystemTimeZoneById("Europe/Oslo");

public static DateTime TilLokal(this DateTimeOffset tid)
    => TimeZoneInfo.ConvertTime(tid, Oslo).DateTime;
```

Bruk IANA-navnet `Europe/Oslo`, ikke Windows-navnet. .NET på Linux støtter begge fra .NET 8, men IANA er det som virker i containeren uten ekstra pakker.

7.4 Registrering

```
builder.Services.AddHttpContextAccessor();
builder.Services.AddScoped<IHusstandContext, DbHusstandContext>();

builder.Services.AddDbContext<DyrepermenDbContext>(opt =>
    opt.UseNpgsql(
        builder.Configuration.GetConnectionString("Postgres"),
        npg => npg.MigrationsAssembly("Dyrepermen.Infrastructure"))
    .UseSnakeCaseNamingConvention());
```

`UseSnakeCaseNamingConvention()` kommer fra pakken `EFCore.NamingConventions` og gjør at C#-egenskapen `VektGram` mapper til kolonnen `vekt_gram` uten manuell konfigurasjon.

8. Forretningslogikk

8.1 Fôrplan

Designprinsipp: applikasjonen anbefaler ikke fôrmengde. Den regner ut den regelen brukeren selv har lagt inn. Dette er et bevisst valg. Riktig mengde avhenger av art, rase, alder, fôrtype, aktivitetsnivå og hold, og varierer dessuten sterkt mellom tørrfôr og råfôr. En innebygd formel ville gitt et tall som ser autoritativt ut uten å ha dekning for det, og appen skal kunne brukes til hund, katt, valp og voksen uten å være spesialisert mot noen av dem.

Metoden lagres som `char(1)` og må mappes eksplisitt — gjetter man på verdiene, brytes CHECK-constrainten ved første lagring:

```
public enum Formetode { Prosent, Gram }

b.Entity<Forplan>()
  .Property(f => f.Metode)
  .HasConversion(
    v => v == Formetode.Prosent ? 'P' : 'G',
    v => v == 'P' ? Formetode.Prosent : Formetode.Gram)
  .HasColumnType("char(1)");
```

Samme mønster gjelder `Dyr.Art` (H/K), `Dyr.Kjonn` (T/H), `Behandling.Type` (V/O/F/K/T), `Dokument.Kategori` (V/J/K/A) og `Handleliste.Status` (A/K).

Brukeren velger én av to metoder:

Metode	Inndata	Typisk bruk
P — prosent av kroppsvekt	Prosentstøt, f.eks. 5,0 %	Råføring, og valper der mengden følger vekten oppover
G — fast mengde	Gram per dag, f.eks. 400	Tørrfôr etter produsentens tabell, eller mengde avtalt med veterinær

Forskjellen som betyr noe: **prosentmetoden er levende**. Den leser siste vektregistrering hver gang, så mengden følger valpen automatisk gjennom vekstfasen uten at noen må huske å justere. Fast mengde står stille til den endres.

```

public sealed class ForplanService
{
    private readonly DyrepermenDbContext _db;

    public async Task<ForplanResultat> BeregnAktiv(
        int dyrId, CancellationToken ct)
    {
        var plan = await _db.Forplan
            .SingleOrDefaultAsync(f => f.DyrId == dyrId && f.Aktiv, ct);

        if (plan is null)
            return ForplanResultat.IngenPlan();

        int gramPerDag;

        if (plan.Metode == Formetode.Prosent)
        {
            var siste = await _db.Vekt
                .Where(v => v.DyrId == dyrId)
                .OrderByDescending(v => v.Dato)
                .ThenByDescending(v => v.Id)
                .FirstOrDefaultAsync(ct);

            if (siste is null)
                return ForplanResultat.ManglerVekt();

            // prosent_tidels = 50 betyr 5,0 %
            gramPerDag = (int)Math.Round(
                siste.VektGram * plan.ProsentTidels!.Value / 1000.0);

            return ForplanResultat.Ok(
                gramPerDag,
                plan.AntallMaltider,
                grunnlagVektGram: siste.VektGram,
                grunnlagDato: siste.Dato);
        }

        gramPerDag = plan.GramPerDag!.Value;
        return ForplanResultat.Ok(gramPerDag, plan.AntallMaltider);
    }
}

```

Fordeling på måltider gjøres med heltallsdivisjon, og resten legges på første måltid slik at summen alltid stemmer:

```

public static int[] FordelPaMaltider(int gramPerDag, int antall)
{
    var basis = gramPerDag / antall;
    var rest = gramPerDag % antall;
    return Enumerable.Range(0, antall)
        .Select(i => basis + (i < rest ? 1 : 0))
        .ToArray();
}

```

Tilstander grensesnittet må håndtere

Tilstand	Visning
Ingen plan lagt inn	Oppfordring til å opprette en, ingen tall
Prosentplan uten vektregistrering	«Registrer en vekt for å regne ut mengden» — ikke 0 gram
Prosentplan med gammel vekt	Vis mengden, men med dato på vektgrunnlaget synlig
Fast mengde	Vis mengden uten vektreferanse

Vektgrunnlaget skal alltid vises sammen med resultatet ved prosentmetoden. Et tall uten synlig grunnlag er et tall ingen tør stole på.

8.2 Fôringslogg

Funksjonen styres **per dyr** via `dyr.foringslogg_aktiv`, ikke per husstand. Hund og katt har sjelden samme fôringsrutine — hunden mates to faste måltider som begge voksne kan ta, katten har tørrfôr stående fremme. Én felles bryter ville tvunget begge inn i samme modell.

Standardverdi, ikke to nivåer. `husstand_innstilling.foringslogg_standard` er kun en malverdi som kopieres inn på dyret når det opprettes. Den overstyres aldri en bryter som allerede står på et dyr. Dette er bevisst: med to virkelige nivåer oppstår spørsmålet «husstand av, dyr på — hva gjelder?», og hvert svar er feil for noen. Med standardverdi finnes ikke spørsmålet.

```
public async Task<Dyr> OpprettDyr(NyttDyr input, CancellationToken ct)
{
    var std = await _db.HusstandInnstilling
        .SingleOrDefaultAsync(i => i.HusstandId == _husstand.HusstandId, ct);

    var dyr = new Dyr
    {
        HusstandId      = _husstand.HusstandId,
        Navn             = input.Navn,
        ChipNr           = input.ChipNr.TomTilNull(),
        RegNrNkk         = input.RegNrNkk.TomTilNull(),
        ForingsloggAktiv = std?.ForingsloggStandard ?? false,
        ForplanAktiv     = std?.ForplanStandard    ?? true
    };

    _db.Dyr.Add(dyr);
    await _db.SaveChangesAsync(ct);
    return dyr;
}
```

Hvor bryteren betjenes: på dyrets egen redigeringsside, ikke på en separat innstillingsside. Bryteren hører til dyret, og det er der brukeren er når spørsmålet melder seg. `/innstillinger` inneholder kun husstandsomfattende valg — standardverdiene for nye dyr og varselinnstillinger.

Konsekvenser i grensesnittet

Sted	Oppførsel
Forsiden	«Sist matet»-kort vises kun for dyr med <code>foringslogg_aktiv = true</code>
Dyrets detaljside	Fôringsfanen skjules helt når bryteren er av
Fôrplan	Egen bryter (<code>forplan_aktiv</code>), uavhengig av fôringsloggen
Bryter slås av	Loggede rader slettes ikke, kun skjules. Slås den på igjen, er historikken der

Autorisasjon: bryteren styrer visning, ikke tilgang. `ForingController` må avvise `POST` mot et dyr der `foringslogg_aktiv = false`, ellers kan en gammel faneside eller et bokmerke skrive til en avslått funksjon:

```
var dyr = await _db.Dyr.SingleOrDefaultAsync(d => d.Id == dyrId, ct);
if (dyr is null || !dyr.ForingsloggAktiv)
    return NotFound();
```

Registrering skal være så nær ett klikk som mulig:

- **Tidspunktet settes automatisk** på serveren ved lagring. Brukeren velger ikke dato eller klokkeslett.
- **Mengde er valgfri**. Er det en aktiv fôrplan, forhåndsutfylles mengde per måltid fra `FordelPaMaltider`. Brukeren kan overstyre eller la den stå tom og bare huke av at det er gjort.
- **Hvem som registrerte lagres**, og forsiden viser «Matet 07:12 av Marius» for hvert dyr. Det er hele poenget med funksjonen.

```
[HttpPost("{dyrId:int}/foring")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Registrer(
    int dyrId, int? mengdeGram, CancellationToken ct)
{
    await _foring.Registrer(new RegistrerForing(
        DyrId: dyrId,
        MengdeGram: mengdeGram,
        BrukerId: User.BrukerId(), ct);

    return RedirectToAction("Index", "Hjem");
}
```

Tidspunkt er bevisst ikke en parameter. Tjenesten setter `DateTimeOffset.UtcNow` selv, slik at klienten ikke kan påvirke det.

Korrigerings i etterkant: en redigeringsvisning lar brukeren justere tidspunktet på en enkelt registrering. Glemmer man å huke av til man kommer hjem om kvelden, er automatikken feil og må kunne overstyres. Skjulte, ikke-redigerbare tidsstempler skaper mer frustrasjon enn de sparer.

Visning av tid: lagre alltid i UTC (`timestampz`), konverter til `Europe/Oslo` i visningslaget. Ellers forskyver alle registreringer seg med én time ved sommertidsomstillingen.

8.3 Påminnelser

Én tjeneste samler alt som forfaller, uavhengig av kilde:

```

public sealed class PaminnelseService
{
    private readonly DyrepermenDbContext _db;

    public async Task<IReadOnlyList<Paminnelse>> ForfallerInnen(
        int husstandId, int dager, CancellationToken ct)
    {
        var grense = DateOnly.FromDateTime(DateTime.UtcNow).AddDays(dager);

        var behandlinger = await _db.Behandling
            .IgnoreQueryFilters()
            .Where(b => b.Dyr.HusstandId == husstandId
                && b.NesteDato != null
                && b.NesteDato <= grense)
            .Select(b => new Paminnelse(
                b.Dyr.Navn, Kilde.Behandling,
                TypeTekst(b.Type), b.NesteDato!.Value))
            .ToListAsync(ct);

        var forsikringer = await _db.Forsikring
            .IgnoreQueryFilters()
            .Where(f => f.Dyr.HusstandId == husstandId
                && f.FornyesDato != null
                && f.FornyesDato <= grense)
            .Select(f => new Paminnelse(
                f.Dyr.Navn, Kilde.Forsikring,
                $"Fornyetelse {f.Selskap}", f.FornyesDato!.Value))
            .ToListAsync(ct);

        return behandlinger.Concat(forsikringer)
            .OrderBy(p => p.Dato)
            .ToList();
    }
}

```

Medisiner behandles separat fordi de gjentas per time, ikke per dato: neste dose beregnes som siste dose + `intervall_timer`.

8.4 Bakgrunnsjobb

```

public sealed class PaminnelseJobb : BackgroundService
{
    private readonly IServiceScopeFactory _scopes;
    private readonly ILogger<PaminnelseJobb> _log;

    protected override async Task ExecuteAsync(CancellationToken ct)
    {
        while (!ct.IsCancellationRequested)
        {
            try
            {
                using var scope = _scopes.CreateScope();
                var varsler = scope.ServiceProvider
                    .GetRequiredService<IVarselSender>();
                await varsler.SendDagligOppsummering(ct);
            }
            catch (Exception ex)
            {
                _log.LogError(ex, "Påminnelsesjobb feilet");
            }

            await Task.Delay(TimeSpan.FromHours(24), ct);
        }
    }
}

```

Fallgruve på gratis hosting: en applikasjon som spinner ned ved inaktivitet kjører ikke bakgrunnsjobber. Løsningen er å eksponere endepunktet `POST /jobb/paminner` beskyttet av en delt hemmelighet i header, og la en ekstern planlegger kalle det daglig. GitHub Actions med `schedule`-trigger holder, og pinger samtidig databasen slik at den ikke går i dvale.

```

name: Daglig påminnelse
on:
  schedule:
    - cron: '0 6 * * *'
  workflow_dispatch:
jobs:
  ping:
    runs-on: ubuntu-latest
    steps:
      - name: Trigger påminnelser
        run: |
          curl -f -X POST "${{ secrets.APP_URL }}/jobb/paminner" \
            -H "X-Jobb-Nokkel: ${{ secrets.JOBB_NOKKEL }}"

```

9. Controllere og ruter

Alle kontrollere er dekorert med `[Authorize]` og `[Route("dyr/{dyrId:int}/...")]` der de hører til et dyr.

Controller	Rute	Handler
KontoController	/logg-inn , /logg-ut , /registrer	[AllowAnonymous] . Eneste offentlige controller
HjemController	/	Index — dashbord etter innlogging, ikke offentlig forside
DyrController	/dyr	Index , Detaljer , Ny , Rediger (inkl. funksjonsbrytere), Deaktiver
VektController	/dyr/{dyrId}/vekt	Index , Registrer , Slett
BehandlingController	/dyr/{dyrId}/behandling	Index , Ny , Rediger , Slett
MedisinController	/dyr/{dyrId}/medisin	Index , Ny , Avslutt , LoggDose
VetbesokController	/dyr/{dyrId}/vetbesok	Index , Ny , Rediger
ForsikringController	/dyr/{dyrId}/forsikring	Index , Ny , Rediger
DokumentController	/dyr/{dyrId}/dokument	Index , LastOpp , Last , Slett
HandlelisteController	/handleliste	Index , Legg , MarkerKjøpt , Slett
ForplanController	/dyr/{dyrId}/forplan	Index , Ny , Rediger , Deaktiver
ForingController	/dyr/{dyrId}/foring	Index , Registrer , RedigerTid , Slett
InnstillingController	/innstillinger	Index , Lagre — standardverdier og varsler
HusstandController	/husstand	Oppsett , Opprett , Index , LeggTilMedlem , AngreInvitasjon , FjernMedlem , Forlat
KontoController (innlogget)	/konto	Profil , EndreEpost , EndrePassord , LastNedData , Slett

Controller-mønster — tynn, all logikk i tjenesten:

```
[Authorize]
[Route("dyr/{dyrId:int}/medisin")]
public class MedisinController : Controller
{
    private readonly IMedisinService _medisin;

    [HttpPost("{medisinId:int}/dose")]
    [ValidateAntiForgeryToken]
    public async Task<IActionResult> LoggDose(
        int dyrId, int medisinId, CancellationToken ct)
    {
        var brukerId = User.BrukerId();
        var resultat = await _medisin.LoggDose(medisinId, brukerId, ct);

        if (!resultat.Ok)
        {
            TempData["Feil"] = resultat.Melding;
            return RedirectToAction(nameof(Index), new { dyrId });
        }

        return RedirectToAction(nameof(Index), new { dyrId });
    }
}
```

LoggDose skal returnere en advarsel dersom forrige dose ble gitt for kort tid siden — det er hele poenget med å ha to brukere på samme konto. Sjekken hører hjemme i tjenesten, ikke i controlleren.

9.1 Endepunkter uten grensesnitt

Tre endepunkter refereres flere steder i dokumentet og spesifiseres her.

GET /helse — brukes av Render til å avgjøre om instansen lever, og av utrullingsjobben som røyktest. `[AllowAnonymous]`, og den skal **ikke** treffe databasen. En helsesjekk som feiler fordi Neon sover, får Render til å restarte en frisk app.

```
app.MapGet("/helse", () => Results.Ok(new
{
    status = "ok",
    versjon = ThisAssembly.InformationalVersion,
    tid = DateTimeOffset.UtcNow
})).AllowAnonymous();
```

POST /jobb/paminner — trigges av GitHub Actions daglig (kapittel 8.4). Beskyttet av en delt hemmelighet i header, ikke av innlogging:

```
app.MapPost("/jobb/paminner", async (
    HttpContext ctx,
    IVarSelSender varsler,
    IConfiguration konfig,
    CancellationToken ct) =>
{
    var oppgitt = ctx.Request.Headers["X-Jobb-Nokkel"].ToString();
    var forventet = konfig["Jobb:Nokkel"];

    if (string.IsNullOrEmpty(forventet)
        || !CryptographicOperations.FixedTimeEquals(
            Encoding.UTF8.GetBytes(oppgitt),
            Encoding.UTF8.GetBytes(forventet)))
        return Results.Unauthorized();

    var antall = await varsler.SendDagligOppsummering(ct);
    return Results.Ok(new { sendt = antall });
}).AllowAnonymous();
```

Bruk `FixedTimeEquals`, ikke `==`. Strengsammenligning avslutter ved første ulike tegn, og forskjellen i svartid kan brukes til å gjette nøkkelen tegn for tegn.

infra/render.yaml — tjenestedefinisjonen:

```
services:
- type: web
  name: dyrepermen
  runtime: docker
  plan: free
  region: frankfurt
  dockerfilePath: infra/Dockerfile
  dockerContext: .
  healthCheckPath: /helse
  autoDeploy: false
  envVars:
    - key: ASPNETCORE_ENVIRONMENT
      value: Production
    - key: TZ
      value: Europe/Oslo
    - key: ConnectionStrings__Postgres
      sync: false
    - key: Jobb__Nokkel
      generateValue: true
```

`autoDeploy: false` er bevisst. Utrulling skal skje fra arbeidsflyten i kapittel 14.5, etter at testene er grønne og migrasjonene er kjørt — ikke ved hver push.

`sync: false` betyr at verdien settes manuelt i Render-dashbordet og aldri havner i repoet.

9.2 Feilhåndtering og logging

Uventede feil fanges ett sted. Ingen `try/catch` i kontrollere for generell feilhåndtering:

```
if (app.Environment.IsDevelopment())
    app.UseDeveloperExceptionPage();
else
{
    app.UseExceptionHandler("/feil");
    app.UseStatusCodePagesWithReExecute("/feil/{0}");
    app.UseHsts();
}
```

`/feil` viser en nøytral side uten stakksporing eller feilmelding fra databasen. En `PostgresException` som lekker til brukeren kan avsløre tabellnavn og constraint-navn.

Forventede feil er ikke unntak. Constraint-brudd på chipnummer, feil passord ved sletting og utløpt invitasjon returneres som resultattyper (kapittel 6.5) og vises i `ModelState`. Unntak reserveres for det som ikke skal kunne skje.

Logging bruker innebygd `ILogger<T>`, ingen ekstra rammeverk i v1. Render fanger `stdout`.

Nivå	Brukes til
Error	Uventede unntak, feilet e-postutsending, feilet bakgrunnsjobb
Warning	Constraint-brudd, avvist jobbnøkkel, samtidighetskonflikt
Information	Innlogging, utlogging, dyr opprettet, medlem lagt til, konto slettet
Debug	Kun lokalt

Aldri logg: passord, jobbnøkkel, tilkoblingsstrengen, invitasjonsmottakerens e-postadresse i klartekst, eller hele `HttpContext`. Logg bruker-ID, ikke e-post.

```
_log.LogInformation("Dyr {DyrId} opprettet i husstand {HusstandId} av {BrukerId}",
    dyr.Id, dyr.HusstandId, brukerId);
```

Bruk strukturert logging med navngitte plassholdere, ikke strenginterpolering. Da kan feltene søkes på senere uten å parse tekst.

10. Grensesnitt: layout, navigasjon og dashboard

10.1 Prinsipp: én layout, ikke to

Applikasjonen har **én** `_Layout.cshtml` og ett sett navigasjonsmarkup. Forskjellen mellom PC og mobil løses med CSS-brytepunkt, ikke med separate maler eller enhetsdeteksjon på serveren.

Begrunnelsen er vedlikehold: to maler betyr at hver nye menylenke må legges inn to steder, og avvik oppstår innen få uker. Serverside-enhetsdeteksjon via `User-Agent` er dessuten upålitelig og bryter ved nettbrett i landskapsmodus og delt vindu på PC.

Brytepunkt: **992 px** (Bootstraps `lg`). Over dette regnes visningen som PC, under som mobil.

Bredde	Navigasjon
≥ 992 px	Fast sidemeny til venstre, alltid synlig, ingen bryter
< 992 px	Topplinje med logo til venstre og hamburgerknapp til høyre, som åpner en skuff fra høyre

10.2 Implementasjon

Bootstrap 5 har `offcanvas-lg`, som gjør nøyaktig dette uten en linje egen JavaScript: elementet oppfører seg som en uttreksskuff under `lg`, og som vanlig innhold i dokumentflyten fra `lg` og oppover. `offcanvas-end` plasserer skuffen på høyre side.

```
<nav class="navbar d-lg-none border-bottom sticky-top bg-body">
  <div class="container-fluid">
    <a class="navbar-brand" href="/">
      
    </a>
    <button class="navbar-toggler ms-auto" type="button"
      data-bs-toggle="offcanvas"
      data-bs-target="#hovedmeny"
      aria-controls="hovedmeny"
      aria-expanded="false"
      aria-label="Åpne meny">
      <span class="navbar-toggler-icon"></span>
    </button>
  </div>
</nav>

<div class="d-flex">
  <div class="offcanvas-lg offcanvas-end sidemeny"
    tabindex="-1" id="hovedmeny"
    aria-labelledby="hovedmenyTittel">

    <div class="offcanvas-header">
      <h2 class="offcanvas-title h6" id="hovedmenyTittel">Meny</h2>
      <button type="button" class="btn-close"
        data-bs-dismiss="offcanvas"
        data-bs-target="#hovedmeny"
        aria-label="Lukk"></button>
    </div>

    <div class="offcanvas-body flex-column p-0">
      <ul class="nav nav-pills flex-column">
        <li class="nav-item">
          <a class="nav-link" href="/" aria-current="page">Dashbord</a>
        </li>
        <li class="nav-item"><a class="nav-link" href="/dyr">Dyrene</a></li>
        <li class="nav-item"><a class="nav-link" href="/handleliste">Handleliste</a></li>
        <li class="nav-item"><a class="nav-link" href="/husstand">Husstand</a></li>
        <li class="nav-item"><a class="nav-link" href="/innstillinger">Innstillinger</a></li>
      </ul>
      <form method="post" action="/logg-ut" class="mt-auto p-3">
        @Html.AntiForgeryToken()
        <button class="btn btn-outline-secondary w-100">Logg ut</button>
      </form>
    </div>
  </div>

  <main class="flex-grow-1 p-3 p-lg-4">
    @RenderBody()
  </main>
</div>
```

```

.sidemeny { width: 260px; }
@media (min-width: 992px) {
  .sidemeny {
    position: sticky;
    top: 0;
    height: 100vh;
    border-right: 1px solid var(--bs-border-color);
    flex-shrink: 0;
  }
}

```

Detaljer som må stemme

- `data-bs-target` må stå på **både** åpne- og lukkeknappen når `offcanvas-lg` brukes. Utelates den på lukkeknappen, lukkes ikke skuffen.
- **Menyen må lukkes ved navigasjon.** Ved vanlig sidelast skjer det av seg selv, men brukes htmx til delvis oppdatering, blir skuffen stående åpen. Lukk den eksplisitt i en `htmx:afterOnLoad`-lytter.
- `aria-current="page"` settes på gjeldende lenke fra `ViewContext.RouteData`, ikke hardkodet.
- **Berøringsmål minst 44 × 44 px** i skuffen. Standard `nav-link`-padding er for lav på mobil — øk til `0.75rem 1rem` under `lg`.
- **Trygg sone på iOS:** `<meta name="viewport" content="width=device-width, initial-scale=1, viewport-fit=cover">` og `padding-bottom: env(safe-area-inset-bottom)` nederst i skuffen, ellers havner utloggingsknappen under hjemindikatoren.
- Bootstrap håndterer fokusfelle og `Esc` i skuffen automatisk. Ikke skriv egen JavaScript for det.

10.3 Dashbord

Landingssiden etter innlogging. Målet er at begge brukerne skal kunne se hva som gjenstår i dag uten å navigere noe sted.

Innhold, i prioritert rekkefølge:

1. **Dyrekort** — ett per aktivt dyr. Navn, bilde, alder, siste vekt med dato. «Sist matet 07:12 av ...» vises kun når `foringslogg_aktiv` er sann for dette dyret. Hurtigknapper: registrer vekt, logg fôring, nytt notat.
2. **Forfaller snart** — behandlinger, medisiner og forsikringsfornyelser innen 14 dager, sortert på dato. Forfalte elementer øverst, tydelig markert.
3. **Handleliste** — de fem øverste aktive punktene med dyrenavn eller «Felles», og et felt for å legge til nytt.

Responsiv oppførsel

Bredde	Oppsett
< 576 px	Én kolonne, kort stablet. Dyrekort først
576-991 px	To kolonner for dyrekort, full bredde på listene
≥ 992 px	To kolonner: dyrekort til venstre, «forfaller snart» og handleliste til høyre

Bruk `row-cols-1 row-cols-sm-2 row-cols-lg-1` på dyrekortene og et `col-lg-8 / col-lg-4`-oppsett på ytternivå. Ingen egne mediespørringer utover det.

Tomtilstander må spesifiseres, ikke improviseres:

Situasjon	Visning
Ingen dyr registrert	Kort med «Legg til ditt første dyr» og knapp
Ingen vekt på et dyr	«Ingen vekt registrert» med lenke, ikke «0 kg»
Ingenting forfaller	«Ingenting forfaller de neste 14 dagene»
Tom handleliste	Kun inntastingsfeltet, ingen tom boks

Ytelse — dette er viktigere enn det ser ut. Dashbordet er den mest besøkte siden, og databasen skalerer til null mellom økter. Et dashbord som gjør ett spørsmål per dyr per seksjon gir tjue rundturer der én holder.

Krav: **høyst fire spørringer totalt**, uansett antall dyr.

```
public async Task<DashbordVm> Hent(CancellationTokent ct)
{
    var idag = DateOnly.FromDateTime(DateTime.UtcNow);
    var grense = idag.AddDays(14);

    var dyr = await _db.Dyr
        .OrderBy(d => d.Navn)
        .Select(d => new DyrKortVm(
            d.Id, d.Navn, d.Art, d.BildeFilnavn,
            d.Fodselsdato,
            d.ForingsloggAktiv,
            d.Vekter.OrderByDescending(v => v.Dato)
                .ThenByDescending(v => v.Id)
                .Select(v => new VektVm(v.VektGram, v.Dato))
                .FirstOrDefault(),
            d.ForingsloggAktiv
                ? d.Foringer.OrderByDescending(f => f.Tidspunkt)
                    .Select(f => new SistMatetVm(
                        f.Tidspunkt, f.GittAv.Visningsnavn))
                    .FirstOrDefault()
                : null))
        .ToListAsync(ct);

    var forfaller = await _paminnerelser.ForfallerInnen(14, ct);

    var handleliste = await _db.Handleliste
        .Where(h => h.Status == 'A')
        .OrderBy(h => h.OpprettetDato)
        .Take(5)
        .Select(h => new HandlelisteVm(
            h.Id, h.Tekst, h.Antall,
            h.Dyr != null ? h.Dyr.Navn : null))
        .ToListAsync(ct);

    return new DashbordVm(dyr, forfaller, handleliste);
}
```

Korrelerte underspørringer i `Select` oversettes av Npgsql til `LEFT JOIN LATERAL` og kjøres i samme rundtur. Bruk aldri `Include` etterfulgt av `.Last()` i C# — da hentes hele vekthistorikken for hvert dyr.

Ikke legg til automatisk oppdatering eller polling. Dashbordet er statisk til brukeren gjør noe. En bakgrunnsforespørsel hvert tiende sekund holder databasen våken og spiser gratiskvoten uten å gi verdi for to brukere.

11. Autentisering, innlogging og oppstart

11.1 Ingen offentlig forside

Applikasjonen har ingen landingsside. Rot-URL-en `/` sender uautentiserte brukere rett til `/logg-inn`. Dette løses med en fallback-policy som gjør autentisering til standard for alt, i stedet for å dekorere hver enkelt controller:

```
builder.Services.AddAuthorization(o =>
{
    o.FallbackPolicy = new AuthorizationPolicyBuilder()
        .RequireAuthenticatedUser()
        .Build();
});
```

Kun `KontoController` (innlogging, registrering, passordtilbakestilling) og helsesjekk-endepunktet merkes `[AllowAnonymous]`. Prinsippet er fail closed: glemmer man å sikre en ny controller, blir den låst — ikke åpen.

Innloggingssiden skal være minimal: logo, e-post, passord, «Husk meg», innloggingsknapp, og en diskret lenke til registrering. Ingen markedsføring, ingen funksjonsoversikt, ingen navigasjonsmeny.

11.2 Vedvarende innlogging — 30 dager med fornyelse

Kravet: brukeren skal maksimalt måtte logge inn én gang i måneden, og hver gang appen brukes skal de 30 dagene starte på nytt. Dette er glidende utløp (`SlidingExpiration`) kombinert med en vedvarende informasjonskapsel.

```
builder.Services.ConfigureApplicationCookie(o =>
{
    o.LoginPath      = "/logg-inn";
    o.LogoutPath     = "/logg-ut";
    o.AccessDeniedPath = "/logg-inn";

    o.ExpireTimeSpan = TimeSpan.FromDays(30);
    o.SlidingExpiration = true;

    o.Cookie.Name      = "dyrepermen_auth";
    o.Cookie.HttpOnly  = true;
    o.Cookie.SecurePolicy = CookieSecurePolicy.Always;
    o.Cookie.SameSite   = SameSiteMode.Lax;
    o.Cookie.IsEssential = true;
});
```

E-post er brukernavnet. Identity bruker `UserName` som innloggingsnavn, ikke `Email`. Sett dem like ved registrering, og krev unik e-post:

```
builder.Services.AddIdentity<Bruker, IdentityRole<int>>(o =>
{
    o.User.RequireUniqueEmail = true;
    o.SignIn.RequireConfirmedAccount = false;
    o.Password.RequiredLength = 10;
    o.Lockout.MaxFailedAccessAttempts = 5;
    o.Lockout.DefaultLockoutTimeSpan = TimeSpan.FromMinutes(15);
})
.AddEntityFrameworkStores<DyrepermenDbContext>()
.AddDefaultTokenProviders();

// Ved registrering:
var bruker = new Bruker { UserName = epost, Email = epost };
```

Gjøres ikke dette, feiler innlogging med e-post selv om passordet er riktig. Ved bytte av e-postadresse må `UserName` oppdateres i samme operasjon, ellers logger brukeren fortsatt inn med den gamle adressen.

Ved innlogging må `isPersistent` settes fra avkryssingsboksen. Uten den blir informasjonskapselen en øtkapsel som dør når nettleseren lukkes, uansett hva `ExpireTimeSpan` sier:

```
var resultat = await _signIn.PasswordSignInAsync(
    input.Epost,
    input.Passord,
    isPersistent: input.HuskMeg,
    lockoutOnFailure: true);
```

Fire ting som må stemme for at dette faktisk virker:

1. Data Protection-nøkklene må overleve omstart. Dette er den viktigste enkeltdetaljen i hele kapittelet. Informasjonskapselen er kryptert med en nøkkel fra Data Protection-nøkkelringen. Som standard lagres den på filsystemet i containeren — og forsvinner ved hver utrulling og hver omstart. Resultatet er at alle blir logget ut hver gang du deployer, og «husk meg» oppleves som ødelagt. På en plattform som spinner ned ved inaktivitet skjer dette flere

ganger i uken.

Løsningen er å lagre nøkkelingen i databasen:

```
builder.Services.AddDataProtection()
    .PersistKeysToDbContext<DyrepermenDbContext>()
    .SetApplicationName("dyrepermen")
    .SetDefaultKeyLifetime(TimeSpan.FromDays(90));
```

Krever pakken `Microsoft.AspNetCore.DataProtection.EntityFrameworkCore` og at `DyrepermenDbContext` implementerer `IDataProtectionKeyContext`:

```
public DbSet<DataProtectionKey> DataProtectionKeys => Set<DataProtectionKey>();
```

`SetApplicationName` må være låst til en fast streng. Standardverdien utledes fra applikasjonens navn på disk, og endres den, blir alle eksisterende informasjonskapsler ugyldige.

Behandle denne strengen som en intern identifikator, ikke som produktnavnet. Den skal aldri endres, heller ikke hvis appen døpes om senere. Skriv det som en kommentar i koden, ellers vil noen «rydde opp» i den ved neste navnebytte og logge ut alle brukere:

```
// Intern nøkkelingidentifikator. Må ALDRI endres – heller ikke ved navnebytte.
// Endring ugyldiggjør alle innloggingskapsler.
.SetApplicationName("dyrepermen")
```

2. Glidende utløp fornyer ikke ved hvert kall. ASP.NET Core skriver bare ny informasjonskapsel når mer enn halve levetiden er brukt. Med 30 dagers vindu betyr det at fornyelse skjer ved bruk etter dag 15, ikke ved hvert sidevisning. Effekten er likevel den ønskede: en bruker som er innom minst én gang hver 30. dag forblir innlogget i det uendelige. Det er verdt å vite når man tester — en fornyelse etter to dager er ikke synlig i informasjonskapselens utløpsdato.

3. Sikkerhetsstempleet revalideres mot databasen. Identity sjekker `SecurityStamp` med jevne mellomrom og logger ut brukeren hvis det er endret. Standardintervallet er 30 minutter. Sett det høyere for å redusere databasetreff på et gratis databasenivå:

```
builder.Services.Configure<SecurityStampValidatorOptions>(o =>
    o.ValidationInterval = TimeSpan.FromHours(12));
```

Passordbytte logger da ut andre enheter innen 12 timer i stedet for 30 minutter. Det er en akseptabel avveining for to brukere, men skal være et bevisst valg.

4. Utlogging må være tilgjengelig. Med 30 dagers vedvarende innlogging trenger brukeren en tydelig måte å logge ut på, særlig på en delt eller lånt enhet. POST `/logg-ut` med antiforgery-token, aldri en GET-lenke.

11.3 Oppstartsskjerm

En overlegg-skjerm med logo, spinner og status tekst vises mens siden laster, og skjules når dokumentet er ferdig. Den ligger i `_Layout.cshtml` og skal rendres før alt annet innhold, slik at den er synlig umiddelbart uten å vente på CSS-filer:

```
<div id="oppstart" role="status" aria-live="polite">
    
    <div class="spinner" aria-hidden="true"></div>
    <p>Laster ...</p>
</div>
```

```
#oppstart {
  position: fixed; inset: 0; z-index: 9999;
  display: flex; flex-direction: column;
  align-items: center; justify-content: center; gap: 1rem;
  background: #fff;
  transition: opacity .25s ease;
}
#oppstart.skjult { opacity: 0; pointer-events: none; }

.spinner {
  width: 32px; height: 32px;
  border: 3px solid #ddd; border-top-color: #555;
  border-radius: 50%;
  animation: snurr .8s linear infinite;
}
@keyframes snurr { to { transform: rotate(360deg); } }

@media (prefers-reduced-motion: reduce) {
  .spinner { animation: none; border-top-color: #ddd; }
}
```

```
window.addEventListener('load', () => {
  const o = document.getElementById('oppstart');
  o.classList.add('skjult');
  setTimeout(() => o.remove(), 300);
});
```

Tre krav til implementasjonen:

- **Stilene må ligge inline i `<head>`**, ikke i en ekstern CSS-fil. Ligger de eksternt, blinker det urenderte innholdet frem før overlegget får stil.
- **Fallback hvis JavaScript feiler.** Overlegget må aldri kunne bli hengende. Legg inn en `setTimeout` på 8 sekunder som fjerner det uansett, og en `<noscript>`-regel som skjuler det.
- **Skjermlesere:** `role="status"` og `aria-live="polite"` gjør at teksten annonseres, og `aria-hidden` på spinneren hindrer at dekorasjonen leses opp.

Ærlig begrensning: dette overlegget hjelper ikke mot kaldstart på gratis hosting. Når containeren er spunnet ned, svarer serveren ikke i det hele tatt, og nettleseren viser en tom side til den våkner — det finnes ingen markup å vise fordi ingenting er levert. Overlegget dekker sideoverganger og innlasting av ressurser, ikke ventetid før første byte. Det eneste som løser kaldstart er å holde instansen varm, og den daglige jobben fra kapittel 8.4 gjør ikke det alene. Vurder å øke pingfrekvensen til hvert 10. minutt hvis ventetiden blir plagsom, eller å gå over til Railway.

12. Husstand, brukerkonto og innstillinger

12.1 Oppsett ved første innlogging

En nyregistrert bruker har `husstand_id = NULL` og kan ikke se noe innhold. Fallback-policyen slipper dem inn i applikasjonen, men et middleware sender dem til `/husstand/oppsett` inntil tilknytningen er på plass.

```

public class KreverHusstandMiddleware
{
    private static readonly string[] Unntak =
        { "/husstand/oppsett", "/logg-ut", "/konto", "/helse" };

    public async Task InvokeAsync(HttpContext ctx, RequestDelegate next)
    {
        var sti = ctx.Request.Path.Value ?? "/";

        if (ctx.User.Identity?.IsAuthenticated == true
            && ctx.User.FindFirst("husstand_id") is null
            && !Unntak.Any(u => sti.StartsWith(u, StringComparison.OrdinalIgnoreCase)))
        {
            ctx.Response.Redirect("/husstand/oppsett");
            return;
        }

        await next(ctx);
    }
}

```

Oppsettsiden gir to valg, likestilt presentert:

- **Opprett ny husstand** — brukeren oppgir et navn. Applikasjonen oppretter `husstand`, en tilhørende `husstand_innstilling` med standardverdier, og setter `bruker.husstand_id`.
- **Bli med i en eksisterende** — vises kun som forklarende tekst: «Er du lagt til av noen andre, skjer det automatisk. Be dem legge til e-postadressen din.» Det finnes ingen kode å taste inn.

12.2 Opprette ny husstand

```

public async Task<int> OpprettHusstand(
    string navn, int brukerId, CancellationToken ct)
{
    await using var tx = await _db.Database.BeginTransactionAsync(ct);

    var husstand = new Husstand { Navn = navn };
    _db.Husstand.Add(husstand);
    await _db.SaveChangesAsync(ct);

    _db.HusstandInnstilling.Add(new HusstandInnstilling
    {
        HusstandId = husstand.Id
    });

    var bruker = await _db.Users.SingleAsync(u => u.Id == brukerId, ct);
    bruker.HusstandId = husstand.Id;

    await _db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);

    await _signIn.RefreshSignInAsync(bruker);
    return husstand.Id;
}

```

To ting som må stemme:

`HusstandInnstilling` må opprettes samtidig, i samme transaksjon. Mangler raden, faller `OpprettDyr` tilbake på hardkodete standardverdier og innstillingssiden krasjer på en `null`-referanse.

`RefreshSignInAsync` er ikke valgfritt. Uten den mangler informasjonskapselen `husstand_id`-claim-en, og query-filtrene returnerer tomt i opptil 30 dager — nøyaktig så lenge den vedvarende innloggingen varer. Dette er den mest sannsynlige

feilen i hele kapittelet, fordi den ser ut som «databasen lagret ikke».

12.3 Legge til medlem med e-post

Modellen er bevisst enkel: et eksisterende medlem taster inn en e-postadresse, og personen er med. Ingen kode, ingen godkjenning fra mottaker, ingen e-postutsending. Dette er akseptabelt fordi husstanden er en privat enhet der medlemmene kjenner hverandre og avtaler tilgangen muntlig.

Adressen kan være i to tilstander, og de håndteres ulikt:

Adressen er allerede registrert som bruker. `husstand_id` settes direkte, og personen ser husstandens innhold ved neste sidevisning.

Adressen finnes ikke ennå. Det lagres en rad i `husstand_invitasjon`. Når noen senere registrerer seg med nøyaktig den adressen, knyttes de automatisk til husstanden under registreringen — uten å taste noe ekstra.

```
public async Task<LeggTilResultat> LeggTilMedlem(
    string epost, int husstandId, int utfortAvBrukerId, CancellationToken ct)
{
    var normalisert = epost.Trim().ToLowerInvariant();

    var eksisterende = await _db.Users
        .SingleOrDefaultAsync(u => u.NormalizedEmail == normalisert.ToUpperInvariant(), ct);

    if (eksisterende is not null)
    {
        if (eksisterende.HusstandId == husstandId)
            return LeggTilResultat.AlleredeMedlem;

        // Kritisk sjekk – se under
        if (eksisterende.HusstandId is not null)
            return LeggTilResultat.TilhorerAnnenHusstand;

        eksisterende.HusstandId = husstandId;
        await _db.SaveChangesAsync(ct);
        return LeggTilResultat.LagtTil;
    }

    _db.HusstandInvitasjon.Add(new HusstandInvitasjon
    {
        HusstandId = husstandId,
        Epost = normalisert,
        OpprettetAvBrukerId = utfortAvBrukerId
    });

    await _db.SaveChangesAsync(ct);
    return LeggTilResultat.VenterPaRegistrering;
}
```

Sjekken mot `eksisterende.HusstandId is not null` er den viktigste linjen i metoden. Uten den kan hvem som helst taste inn e-postadressen til en fremmed bruker og flytte dem ut av deres egen husstand, siden `husstand_id` er enkeltverdi. Den forrige husstanden mister et medlem uten varsel, og er det siste medlem, blir alle dataene utilgjengelige. Dette er ikke et teoretisk scenario — det er den forventede oppførselen hvis sjekken mangler.

Feilmeldingen skal være nøytral: «Denne adressen kan ikke legges til nå.» Å svare «personen tilhører allerede en husstand» bekrefter for en fremmed at adressen er registrert i systemet.

Ved registrering slås ventende invitasjoner opp på adressen:

```

var invitasjon = await _db.HusstandInvitasjon
    .SingleOrDefaultAsync(i => i.Epost == normalisert
        && i.InnlostTid == null, ct);

if (invitasjon is not null)
{
    bruker.HusstandId = invitasjon.HusstandId;
    invitasjon.InnlostAvBrukerId = bruker.Id;
    invitasjon.InnlostTid = DateTimeOffset.UtcNow;
}

```

Den nye brukeren hopper da over `/husstand/oppsett` og lander rett på dashbordet med husstandens dyr.

Angre: ventende invitasjoner vises i medlemslisten som «venter på registrering», med en knapp for å slette raden. En invitasjon som er løst inn, kan ikke angres — der bruker man «fjern medlem» i stedet.

12.3.1 Følgefeil: claim-en blir foreldet

Å sette `husstand_id` på en annen brukers rad løser ikke problemet alene. Den personen har sin egen informasjonskapsel med `husstand_id`-claim-en fra sist de logget inn — og den er tom. Serveren kan ikke kalle `RefreshSignInAsync` på en annen brukers økt.

Med vedvarende innlogging i 30 dager betyr det at personen kan bli lagt til i husstanden og fortsatt se en tom applikasjon i ukevis.

Løsningen er å slutte å lese `husstand_id` fra claim-en. Hent den fra databasen én gang per forespørsel i stedet:

```

public class DbHusstandContext : IHusstandContext
{
    private readonly IHttpContextAccessor _accessor;
    private readonly DyrepermenDbContext _db;
    private int? _bufret;

    public int HusstandId
    {
        get
        {
            if (_bufret is not null) return _bufret.Value;

            var brukerId = _accessor.HttpContext?.User.BrukerId();
            if (brukerId is null) return _bufret = 0 ?? 0;

            _bufret = _db.Users
                .Where(u => u.Id == brukerId)
                .Select(u => u.HusstandId ?? 0)
                .FirstOrDefault();

            return _bufret.Value;
        }
    }
}

```

Tjenesten er registrert `Scoped`, så oppslaget skjer én gang per forespørsel uansett hvor mange query-filtre som leser den. Kostnaden er ett indeksert primærnøkkeloppslag — ubetydelig sammenlignet med en klasse av feil som ellers ser ut som «databasen lagret ikke».

Dette erstatter claim-baserte oppslag i kapittel 7.2, og gjør samtidig `RefreshSignInAsync`-kravet i 12.2 overflødig. Behold likevel kallet ved opprettelse av egen husstand — det koster ingenting og gjør oppførselen forutsigbar hvis noen senere legger claim-en tilbake.

Fjerne et medlem: ethvert medlem kan fjerne et annet, siden alle er likestilte. Handlingen setter `husstand_id = NULL` og krever bekreftelse. Den fjernede beholder kontoen sin og havner på `/husstand/oppsett` ved neste sidevisning. Data om dyrene blir værende i husstanden.

Forlate husstand: samme operasjon, utført på seg selv. Er du siste medlem, må du velge: slett husstanden med alt innhold, eller legg til noen først. Applikasjonen skal ikke tillate en husstand uten medlemmer — da blir dataene utilgjengelige for alltid uten å bli slettet.

12.4 Innstillinger

`/innstillinger` er én side med tydelig atskilte seksjoner, ikke en undermeny med mange nivåer.

Seksjon	Innhold
Min profil	Visningsnavn, e-post, endre passord
Husstand	Husstandsnavn, medlemsliste, legg til medlem på e-post, ventende invitasjoner
Standardverdier	<code>foringslogg_standard</code> , <code>forplan_standard</code> for nye dyr
Varsler	<code>varsler_aktiv</code> , hvor mange dager før forfall
Data	Last ned alle data som JSON
Faresone	Forlat husstand, slett konto permanent

Endre e-post krever bekreftelse på den nye adressen før byttet trer i kraft. Identity har `GenerateChangeEmailTokenAsync` og `ChangeEmailAsync` for dette. Uten bekreftelse kan en tastefeil låse brukeren ute permanent, siden e-post er innloggingsnavnet.

Endre passord oppdaterer `SecurityStamp`, som logger ut andre enheter. Merk samspillet med kapittel 11.2: `ValidationInterval` er satt til 12 timer, så utloggingen på andre enheter kan ta inntil 12 timer. Skal passordbytte virke umiddelbart overalt, må intervallet settes lavere — det er en bevisst avveining mot databasetreff, og teksten ved passordskjemaet skal si hva som faktisk skjer.

Faresonen skilles visuelt fra resten og plasseres nederst. Ingen destruktiv handling skal ligge ved siden av en lagre-knapp.

12.5 Slette bruker permanent

Dette er den vanskeligste operasjonen i applikasjonen, fordi to ting som ser like ut skal behandles helt ulikt:

- **Personopplysninger** — navn, e-post, passordhash, innloggingsspor. Skal slettes.
- **Husstandens data** — vektlogger, føringer, behandlinger, dyr. Tilhører husstanden, ikke personen. Skal bestå, men avidentifiseres.

En kaskadesletting fra brukeren ville tatt med seg hele vekthistorikken til hunden fordi det tilfeldigvis var denne personen som registrerte målingene. Det er feil, og det er ikke til å reversere.

Regelen: slett brukerraden, sett `*_av_bruker_id = NULL` på alt vedkommende har registrert. Visningslaget skriver «slettet bruker» der navnet sto.

```

public async Task<SlettResultat> SlettBruker(
    int brukerId,
    string passord,
    bool bekreftetSletteHusstand,
    CancellationToken ct)
{
    var bruker = await _db.Users.SingleAsync(u => u.Id == brukerId, ct);

    if (!await _brukere.CheckPasswordAsync(bruker, passord))
        return SlettResultat.FeilPassord;

    await using var tx = await _db.Database.BeginTransactionAsync(ct);

    var husstandId = bruker.HusstandId;
    var sisteMedlem = husstandId is not null
        && await _db.Users.CountAsync(u => u.HusstandId == husstandId, ct) == 1;

    if (sisteMedlem && !bekreftetSletteHusstand)
        return SlettResultat.MaBekrefteHusstandsletting;

    // ON DELETE SET NULL håndterer aidentifiseringen i databasen
    await _brukere.DeleteAsync(bruker);

    if (sisteMedlem)
        await _db.Husstand
            .Where(h => h.Id == husstandId)
            .ExecuteDeleteAsync(ct);

    await _db.SaveChangesAsync(ct);
    await tx.CommitAsync(ct);
    await _signIn.SignOutAsync();

    return SlettResultat.Ok;
}

```

Krav til flyten:

Krav	Begrunnelse
Passord må testes på nytt	Enheten kan stå ulåst. Vedvarende innlogging i 30 dager gjør dette viktigere, ikke mindre viktig
Eksplisitt bekreftelse, ikke bare «Er du sikker?»	La brukeren skrive husstandsnavnet eller ordet SLETT
Nedlasting av data tilbys før sletting	Én knapp som eksporterer alt som JSON. Kostnaden er lav, angrefristen er null
Siste medlem varsles særskilt	«Du er eneste medlem. Sletting fjerner også husstanden og alle data om N dyr.» Vis antallet
Umiddelbar utlogging	Informasjonskapselen er gyldig i 30 dager til og må ugyldiggjøres
Ingen angrefrist	Sletting er endelig. Si det tydelig i stedet for å antyde at noe kan gjenopprettes

Ikke bygg myk sletting for brukerkontoer. En `slettet`-kolonne som beholder e-post og passordhash er ikke sletting, og det er verre enn ingenting: brukeren tror opplysningene er borte mens de ligger der. Enten sletter du raden, eller så er den ikke slettet.

Dyr er annerledes, og der er myk sletting riktig — se kapittel 5. Forskjellen er at et dyr ikke er en person med krav på sletting, og at historikken om et dyr som er gått bort har verdi for husstanden.

12.6 Rollemodell

Alle medlemmer av en husstand er likestilte i v1. Ingen lese-/skrivendifferensiering, ingen eier eller administrator. Det er to

voksne som deler ansvaret, ikke en organisasjon.

Konsekvensen er at hvem som helst kan fjerne hvem som helst, og slette husstanden hvis de er alene igjen. Det er akseptabelt for målgruppen, men det er en bevisst begrensning — ikke noe som er glemt. Skal appen senere brukes av et bofellesskap eller en klinikk, må rollemodellen inn før brukerantallet vokser.

13. Hosting

13.1 Vurderte alternativer

Plattform	Gratis?	Egnethet for .NET	Vurdering
Render	Ja, permanent	Docker	Web-tjeneste med 512 MB RAM, kaldstart etter inaktivitet, ikke kredittkort. Nærmeste ekte gratistilbud i 2026.
Railway	Nei, ca. \$5/mnd	Auto-oppdager .NET	Best utvikleropplevelse. 30 dagers prøveperiode med kreditt.
Fly.io	Nei	Docker	Gratisnivået ble avviklet i 2024. Nye brukere får kun 2 VM-timer eller 7 dager.
Azure App Service F1	Ja, begrenset	Førsteklasses	God .NET-integrasjon, men F1-nivået er stramt på CPU-kvote.
Koyeb	Ja, med grenser	Docker	Fungerer, men mindre moden dokumentasjon for .NET.

13.2 Database

Tjeneste	Gratis	Kritisk detalj
Neon	0,5 GB per prosjekt, opptil 100 prosjekter, 100 CU-timer/mnd	Skalerer til null etter 5 minutters inaktivitet, gjenopptas på under ett sekund. Databasen pauses aldri permanent.
Supabase	500 MB, 2 prosjekter, 50 000 MAU	Pauses helt etter 7 dager uten aktivitet og må gjenopprettes manuelt fra dashbordet. Ingen automatiske sikkerhetskopier på gratisnivå.

13.3 Anbefaling

Render (webtjeneste) + Neon (PostgreSQL).

Begrunnelsen er 7-dagersregelen hos Supabase. Dette er en familieapp som kan gå en uke uten at noen logger inn — nøyaktig bruksmønsteret som utløser pausen. Neons skalering til null gir samme kostnad (null) uten den fellen. Supabase' tilleggstjenester (Auth, Storage, Realtime) er dessuten irrelevante her, siden ASP.NET Core Identity dekker autentisering.

Oppgraderingsvei når kaldstartene blir irriterende: **Railway til ca. \$5/mnd** med app og database i samme prosjekt. Det er terskelen der betaling faktisk kjøper noe merkbart.

Hvis Supabase likevel velges: sett opp en GitHub Actions-jobb som pinger databasen daglig. Den samme jobben som trigger påminnelsene dekker dette.

13.4 Tilkobling

Neon krever SSL og bruker en pooler-endepunkt. Tilkoblingsstrengen legges i miljøvariabel, aldri i `appsettings.json`:

```
Host=ep-xxx-pooler.eu-central-1.aws.neon.tech;
Database=dyrepermen;
Username=dyrepermen_owner;
Password=***;
SSL Mode=Require;
Trust Server Certificate=true;
Maximum Pool Size=10
```

Sett `Maximum Pool Size` lavt. Gratisnivåene har begrenset antall samtidige tilkoblinger, og en enkelt containerinstans trenger ikke mer enn ti.

14. Containerisering og utrulling

14.1 Dockerfile

Ligger i `infra/Dockerfile`. Byggekonteksten er repo-roten — se kapittel 3.5.

```
FROM mcr.microsoft.com/dotnet/sdk:9.0 AS build
WORKDIR /src
COPY ["src/Dyrepermen.Web/Dyrepermen.Web.csproj", "Dyrepermen.Web/"]
COPY ["src/Dyrepermen.Application/Dyrepermen.Application.csproj", "Dyrepermen.Application/"]
COPY ["src/Dyrepermen.Domain/Dyrepermen.Domain.csproj", "Dyrepermen.Domain/"]
COPY ["src/Dyrepermen.Infrastructure/Dyrepermen.Infrastructure.csproj", "Dyrepermen.Infrastructure/"]
RUN dotnet restore "Dyrepermen.Web/Dyrepermen.Web.csproj"
COPY src/ .
RUN dotnet publish "Dyrepermen.Web/Dyrepermen.Web.csproj" \
    -c Release -o /app/publish /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:9.0 AS final
WORKDIR /app
ENV ASPNETCORE_URLS=http://+:8080
ENV DOTNET_gcServer=0
EXPOSE 8080
COPY --from=build /app/publish .
ENTRYPOINT ["dotnet", "Dyrepermen.Web.dll"]
```

`DOTNET_gcServer=0` er viktig på 512 MB RAM — server-GC allokere per kjerne og spiser opp minnet på små instanser.

`Directory.Build.props` og `Directory.Packages.props` må kopieres inn før `dotnet restore`, ellers finner ikke restore pakkeversjonene:

```
COPY Directory.Build.props Directory.Packages.props global.json ./
```

Denne linjen legges rett før `COPY`-linjene for `.csproj`-filene. Utelates den, feiler bygget med at pakker mangler versjon — en feilmelding som ikke peker mot årsaken.

14.2 Lokal kjøring med Docker Compose

Compose-filen dekker to bruksmåter, styrt med profiler. Begge ligger i samme fil så de ikke kommer i utakt.

Daglig utvikling: kun databasen i container, appen kjøres med `dotnet run`. Du beholder hot reload, debugger og raske omstarter.

Verifisering før utrulling: hele stakken i container. Dette er eneste måten å teste at Dockerfile-en faktisk virker før Render prøver den.

```

name: dyrepermen

services:
  db:
    image: postgres:16-alpine
    environment:
      POSTGRES_DB: dyrepermen
      POSTGRES_USER: dyrepermen
      POSTGRES_PASSWORD: ${POSTGRES_PASSWORD:-utvikling}
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U dyrepermen -d dyrepermen"]
      interval: 5s
      timeout: 3s
      retries: 10

  web:
    profiles: ["full"]
    build:
      context: ..
      dockerfile: infra/Dockerfile
    environment:
      ASPNETCORE_ENVIRONMENT: Development
      ConnectionStrings__Postgres: >-
        Host=db;Port=5432;Database=dyrepermen;
        Username=dyrepermen;Password=${POSTGRES_PASSWORD:-utvikling};
        Maximum Pool Size=10
      Jobb__Nokkel: ${JOB__NOKKEL:-lokal-nokkel}
      TZ: Europe/Oslo
    ports:
      - "8080:8080"
    depends_on:
      db:
        condition: service_healthy

volumes:
  pgdata:

```

Kommandoer

```

# Kun database – daglig utvikling
docker compose -f infra/compose.yaml up -d db
dotnet run --project src/Dyrepermen.Web

# Hele stakken – verifisering før utrulling
docker compose -f infra/compose.yaml --profile full up --build

# Stopp og behold data
docker compose -f infra/compose.yaml down

# Stopp og slett databasen
docker compose -f infra/compose.yaml down -v

```

Detaljer som må stemme

`context: ..` peker på repo-roten, mens `dockerfile: infra/Dockerfile` er relativ til den konteksten. Compose løser stier relativt til compose-filens plassering, ikke til der du står når du kjører kommandoen. Settes `context: ..`, leter bygget etter

.csproj -filene inne i `infra/` og feiler.

`Host=db` — tjenestenavnet er vertsnavnet inne i compose-nettverket. Bruker du `localhost` i tilkoblingsstrengen til `web` -tjenesten, peker den på containeren selv.

`condition: service_healthy` er ikke valgfritt. Uten helsesjekken starter appen før Postgres tar imot tilkoblinger, og første forespørsel feiler med en tilkoblingsfeil som ser ut som feil konfigurasjon.

TZ: `Europe/Oslo` sammen med `InvariantGlobalization=false` (kapittel 3.3) er det som gjør at føringstidspunkter vises riktig lokalt. Mangler en av dem, kaster tidssonekonverteringen i containeren selv om den virker på maskinen din.

Migrasjoner kjøres ikke automatisk. Etter `up -d db` må skjemaet opprettes én gang:

```
dotnet ef database update \  
  --project src/Dyrepermen.Infrastructure \  
  --startup-project src/Dyrepermen.Web
```

Dette gjelder også ved `--profile full` — appen migrerer ikke ved oppstart, i tråd med kapittel 14.3.

Lokale hemmeligheter legges i `infra/.env`, som `.gitignore` må dekke:

```
POSTGRES_PASSWORD=utvikling  
JOBB_NOKKEL=lokal-nokkel
```

Compose leser `.env` fra samme mappe som compose-filen automatisk. Passordet her er kun for lokal utvikling og skal aldri gjenbrukes mot Neon.

14.3 Migrasjoner

Ikke kjør `Database.Migrate()` ved oppstart. Generer et idempotent skript i CI og kjør det som eget steg:

```
dotnet ef migrations script --idempotent \  
  --project src/Dyrepermen.Infrastructure \  
  --startup-project src/Dyrepermen.Web \  
  --output migrations.sql  
  
psql "$DATABASE_URL" -f migrations.sql
```

Dette gir kontroll over når skjemaendringer skjer, og unngår at to instanser migrerer samtidig.

14.4 Hemmeligheter

Variabel	Innhold
<code>ConnectionStrings__Postgres</code>	Neon-tilkoblingsstreng
<code>Jobb__Nokkel</code>	Delt hemmelighet for jobb-ende punkt
<code>Epost__Smtphost</code> , <code>Epost__Bruker</code> , <code>Epost__Passord</code>	SMTP for varsler

Lokalt brukes `dotnet user-secrets`. I produksjon settes de som miljøvariabler i Render-dashbordet. Dobbel understrek mapper til næstet konfigurasjon i .NET.

14.5 CI/CD med GitHub Actions

Alt kjøres i GitHub. Ingenting rulles ut fra en utviklermaskin.

Rekkefølgen er ikke valgfri: bygg → test → migrer → rull ut → røyktest. Rulles appen ut før migrasjonene er kjørt, møter den nye koden et gammelt skjema.

`.github/workflows/bygg.yml`

Kjører på hver push og hver pull request.

```

name: Bygg og test

on:
  push:
    branches: [main]
    paths: ['src/**', 'tests/**', 'infra/**', 'Directory.*.props', 'global.json']
  pull_request:
    branches: [main]

jobs:
  bygg:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-dotnet@v4
        with:
          global-json-file: global.json

      - name: Gjenopprett
        run: dotnet restore

      - name: Bygg
        run: dotnet build --no-restore --configuration Release

      - name: Enhetstester
        run: >-
          dotnet test tests/Dyrepermen.Application.Tests
          --no-build --configuration Release
          --logger "trx;LogFileName=enhet.trx"

      - name: Integrasjonstester
        run: >-
          dotnet test tests/Dyrepermen.Integration.Tests
          --no-build --configuration Release
          --logger "trx;LogFileName=integrasjon.trx"

      - name: Publisert testresultat
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: testresultat
          path: '**/*.trx'

```

Integrasjonstestene trenger ingen tjenestedefinisjon i arbeidsflyten. Testcontainers starter Postgres selv, og `ubuntu-latest` har Docker tilgjengelig.

`--no-build` i testtrinnene forutsetter at bygget over brukte samme konfigurasjon. Utelates `--configuration Release` ett sted, bygges alt på nytt i Debug og tar dobbelt så lang tid.

`.github/workflows/rull-ut.yml`

```

name: Rull ut

on:
  workflow_run:
    workflows: ["Bygg og test"]
    branches: [main]
    types: [completed]
  workflow_dispatch:

concurrency:
  group: produksjon
  cancel-in-progress: false

jobs:
  rull-ut:
    if: github.event.workflow_run.conclusion == 'success'
    runs-on: ubuntu-latest
    environment: produksjon
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-dotnet@v4
        with:
          global-json-file: global.json

      - name: Installer EF-verktøy
        run: dotnet tool install --global dotnet-ef

      - name: Generer idempotent migrasjonsskript
        run: >-
          dotnet ef migrations script --idempotent
          --project src/Dyrepermen.Infrastructure
          --startup-project src/Dyrepermen.Web
          --output migrations.sql

      - name: Kjør migrasjoner
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL }
        run: psql "$DATABASE_URL" -v ON_ERROR_STOP=1 -f migrations.sql

      - name: Utløs utrulling
        run: curl -fsS -X POST "${ secrets.RENDER_DEPLOY_HOOK }"

      - name: Vent og røyktest
        run: |
          for i in $(seq 1 30); do
            if curl -fsS "${ secrets.APP_URL }/helse" > /dev/null; then
              echo "Oppe etter $i forsøk"; exit 0
            fi
            sleep 10
          done
          echo "Appen svarte ikke innen fem minutter"; exit 1

```

concurrency med cancel-in-progress: false hindrer at to utrullinger kjører migrasjoner samtidig. Avbrytes en migrasjon halvveis, står databasen i en tilstand ingen har designet.

ON_ERROR_STOP=1 er kritisk. Uten den fortsetter `psql` etter en feilet setning og rapporterer suksess, mens skjemaet er halvferdig.

Røyktesten tåler kaldstart. Gratisinstansen bruker opptil et minutt på å starte. Uten løkken feiler utrullingene på en app som egentlig er frisk.

`.github/workflows/paminnelser.yml`

```
name: Daglige påminnelser

on:
  schedule:
    - cron: '0 5 * * *'
  workflow_dispatch:

jobs:
  trigger:
    runs-on: ubuntu-latest
    steps:
      - name: Trigg påminnelser
        run: |
          curl -fsS --retry 5 --retry-all-errors --retry-delay 20 \
            -X POST "${{ secrets.APP_URL }}/jobb/paminnelser" \
            -H "X-Jobb-Nokkel: ${{ secrets.JOBB_NOKKEL }}"
```

`cron` i GitHub Actions er UTC. `0 5` gir 07:00 norsk sommertid og 06:00 vintertid. Det er godt nok for en daglig oppsummering — vil du ha fast lokal tid, må du kjøre hver time og filtrere i applikasjonen.

`--retry-all-errors` med 20 sekunders mellomrom dekker kaldstart. Første kall vekker instansen og feiler ofte.

`.github/workflows/sikkerhetskopi.yml`

```
name: Sikkerhetskopi

on:
  schedule:
    - cron: '0 3 * * 0'
  workflow_dispatch:

jobs:
  dump:
    runs-on: ubuntu-latest
    steps:
      - name: Ta dump
        env:
          DATABASE_URL: ${ secrets.DATABASE_URL }
        run: |
          pg_dump "$DATABASE_URL" --no-owner --no-acl \
            | gzip > dyrepermen-$(date +%Y%m%d).sql.gz

      - uses: actions/upload-artifact@v4
        with:
          name: sikkerhetskopi
          path: '*.sql.gz'
          retention-days: 90
```

Artefakter slettes etter 90 dager. Vil du ha lengre oppbevaring, må dumpen krypteres og legges i ekstern lagring — artefakter i et privat repo er ikke offentlig tilgjengelige, men de er ikke sikkerhetskopi i egentlig forstand.

Hemmeligheter i GitHub

Navn	Innhold
DATABASE_URL	Neon-tilkoblingsstreng i postgres:// -format for postgresql
RENDER_DEPLOY_HOOK	Utrullings-URL fra Render-dashbord
APP_URL	https://dyrepermen.onrender.com
JOBBS_NOKKEL	Samme verdi som Jobb_Nokkel i Render

DATABASE_URL bruker URL-format for postgresql, mens applikasjonen bruker nøkkel/verdi-format for Npgsql. De er ikke utbyttable — det er to representasjoner av samme tilkobling, og en agent som gjenbruker den ene i den andre får en feilmelding som ikke forklarer noe.

Grenbeskyttelse: slå på «Require status checks to pass» for main med Bygg og test som påkrevd sjekk. Uten det kan en push med røde tester utløse utrulling.

15. Sikkerhet og personvern

- **Antiforgery** på alle POST-handlinger. [ValidateAntiForgeryToken] uten unntak.
- **Filopplasting:** valider MIME-type og filendelse mot en hvitliste (pdf, jpg, png), maksimalt 10 MB, lagre med generert GUID-navn. Serveres via en controller-handling som verifiserer husstandstilhørighet — aldri direkte fra wwwroot.
- **Autorisasjon på ressursnivå:** query-filteet hindrer at man leser andres data, men verifiser eksplisitt eierskap før skrivning og sletting. Et globalt filter beskytter ikke mot _db.Vekt.Find(id) med IgnoreQueryFilters().
- **HTTPS** tvunget via UseHttpsRedirection() og HSTS i produksjon.
- **Rate limiting** på innlogging via AddRateLimiter — enkelt å legge til, hindrer brute force. Kombiner med Identitys lockoutOnFailure: true.
- **Data Protection-nøkler i database** — se kapittel 11.2. Uten dette blir alle logget ut ved hver utrulling, og «husk meg» virker ikke i praksis.
- **Vedvarende innlogging i 30 dager** er et bevisst valg for en privat husstandsapp. Konsekvensen er at en ulåst enhet på avveie gir tilgang til dataene. Utloggingsknappen må derfor være lett å finne, ikke gjemt i en meny.
- **Sikkerhetskopier:** verken Neon eller Supabase gir automatiske backuper på gratisnivå. Sett opp en ukentlig pg_dump via GitHub Actions som lagrer dumpen kryptert som artefakt eller i egen lagring. Dette er den største reelle risikoen i hele oppsettet.
- **Kontosletting** — se kapittel 12.5. Personopplysninger slettes, husstandens data avidentifiseres. Ikke bygg myk sletting for brukerkontoer.
- **Tillegging av medlem** skjer på e-postadresse uten godkjenning fra mottaker. Sjekken mot at adressen allerede tilhører en annen husstand er ikke kosmetisk — uten den kan hvem som helst tømme en fremmed husstand ved å taste inn adressen deres. Se kapittel 12.3.
- **Personopplysninger:** applikasjonen lagrer navn og e-post på to voksne. Det er innenfor rent personlig bruk, men unngå å legge inn tredjeparters opplysninger (veterinærens navn er greit, kundenummer er unødvendig).

16. Implementeringsfaser og akseptansekriterier

Hver fase ender i en fungerende, utrullet applikasjon med grønne tester. **Gå ikke videre før alle kriteriene under er oppfylt** — de er definisjonen av ferdig, ikke en ønskeliste.

Fase 1 — fundament (MVP)

Monorepo-oppsett med global.json, Directory.Build.props, Directory.Packages.props og .editorconfig før noe annet. Deretter solution-struktur, Docker Compose for lokal Postgres, EF Core, Identity med vedvarende innlogging og Data Protection-nøkler i database, innloggingsside som eneste inngang, oppstartsskjerm, husstandsmodell med query-filtre, Dyr -CRUD.

Ferdig når:

- docker compose -f infra/compose.yaml up -d db etterfulgt av dotnet ef database update gir et komplett skjema uten feil
- dotnet build gir null advarsler (TreatWarningsAsErrors er på)

- `/` uten innlogging omdirigerer til `/logg-inn`. Ingen annen side er tilgjengelig uautentisert
- En bruker kan registrere seg, opprette husstand, og logge inn med e-post og passord
- Med «Husk meg» avkrysset: brukeren er fortsatt innlogget etter at containeren er startet på nytt. Dette verifiserer Data Protection-nøkler i database og er det enkeltkriteriet som oftest ryker
- To brukere i samme husstand ser de samme dyrene
- Filterprøven i kapittel 17 er grønn
- Isolasjonstesten mellom to husstander er grønn
- Duplikat chipnummer avvises med forståelig melding, ikke med en `PostgresException` i nettleseren

Fase 1b — skall og navigasjon (MVP)

Responsiv `_Layout` med sidemeny på PC og høyreskuff på mobil, dashbord med dyrekort og tomtilstander. Gjøres før innholdssidene, slik at hver ny side får riktig ramme fra start.

Ferdig når:

- Ved 1280 px vises sidemenyen fast til venstre, uten hamburgerknapp
- Ved 390 px vises topplinje med hamburger til høyre, og skuffen kommer inn fra høyre
- Skuffen lukkes med `Esc` og ved klikk utenfor, og fokus er fanget mens den er åpen
- Alle menyelementer er minst 44 px høye på mobil
- Dashbordet viser riktig tomtilstand ved null dyr, og «Ingen vekt registrert» framfor 0 kg
- Dashbordet gjør høyst fire databasespørringer med tre dyr registrert. Verifiseres med EF Core-logging på `Information`

Fase 2 — kjernelogging (MVP)

Vekt og behandling med neste dato. Dashbordet viser siste vekt og kommende behandlinger.

Ferdig når:

- Vekt registreres i kilo med komma som desimalskilletegn og lagres som gram. `27,4` gir 27400
- Vekthistorikk vises i synkende datorekkefølge
- Behandling med `NesteDato` innen 14 dager vises på dashbordet, forfalte øverst
- Enhetstest dekker konvertering kilo/gram begge veier, inkludert avrunding
- Appen er rullet ut på Render og `/helse` svarer

MVP er ferdig her. Alt under er senere faser.

Fase 3 — medisin og fôr

Medisiner med doselogg og dobbeltdoseringsvarsel. Fôrplan med begge metoder, koblet mot siste vektregistrering.

Ferdig når: prosentplan gir riktig gram ved kjent vekt, plan uten vektgrunnlag gir «Registrer en vekt» og ikke 0, måltidsfordeling summerer eksakt til dagsmengden, og dose logget for under intervallet gir advarsel.

Fase 4 — påminnelser

`PaminnelseService`, jobb-endeppunkt, GitHub Actions-planlegger, e-postutsending.

Ferdig når: `POST /jobb/paminner` uten gyldig nøkkel gir 401, med gyldig nøkkel sender e-post, og den planlagte arbeidsflyten har kjørt vellykket minst én gang.

Fase 5 — dokumenter og forsikring

Filoplasting, forsikringsregister med fornyelsesvarsel.

Ferdig når: kun pdf, jpg og png under 10 MB aksepteres, filer serveres gjennom en controller som verifiserer husstand, og direkte URL til en annen husstands fil gir 404.

Fase 6 — handleliste

Delt liste på husstands nivå med valgfri kobling til dyr, htmx for avkryssing uten sidelast.

Ferdig når: punkt uten dyr vises som «Felles», avkryssing skjer uten full sidelast, og skuffen på mobil lukkes ved navigasjon.

Fase 6b — fôringslogg

Funksjonsbrytere per dyr, standardverdier på husstands nivå, føringslogg bak bryter.

Ferdig når: bryteren av skjuler fanen og `POST` mot endepunktet gir 404, tidspunkt settes på server og kan ikke sendes fra klient, og «sist matet» viser riktig lokal tid over sommertidsskiftet.

Fase 6c — husstand og konto

Oppsettflyt, tillegging av medlem på e-post, innstillingsside, dataeksport, kontosletting.

Ferdig når: en e-post som tilhører en annen husstand avvises, en ny bruker som registrerer seg med forhåndsgodkjent adresse lander rett på dashbordet, og sletting av bruker beholder vektradene med `NULL` i `registrert_av_bruker_id`.

Fase 7 — polering

Vektgraf, responsiv gjennomgang, sikkerhets kopi-jobb, eventuell PWA-vurdering.

17. Testing

17.1 Omfang og verktøy

Nivå	Verktøy	Dekker
Enhet	xUnit	Ren logikk uten database: førberegning, måltidsfordeling, kilo/gram, dobbeltdoseringsregel, kodenormalisering
Integrasjon	xUnit + Testcontainers (PostgreSQL)	Query-filtre, migrasjoner, constraints, kaskader, samtidighet
Manuell	—	Responsivt oppsett i faktisk nettleser, e-postutsending, filopplasting

Ingen mocking av `DbContext`. InMemory-provideren håndhever verken constraints, partielle unike indekser eller `char(1)`-konvertering, og gir grønne tester på kode som feiler i produksjon. Integrasjonstester kjører mot ekte PostgreSQL i container.

17.2 Testoppsett

```

public sealed class DatabaseFixture : IAsyncLifetime
{
    private readonly PostgreSQLContainer _container =
        new PostgreSQLBuilder()
            .WithImage("postgres:16-alpine")
            .WithDatabase("dyrepermen_test")
            .Build();

    public string Tilkobling => _container.GetConnectionString();

    public async Task InitializeAsync()
    {
        await _container.StartAsync();
        await using var db = LagContext(husstandId: 0);
        await db.Database.MigrateAsync();
    }

    public DyrepermenDbContext LagContext(int husstandId)
    {
        var opt = new DbContextOptionsBuilder<DyrepermenDbContext>()
            .UseNpgsql(Tilkobling)
            .UseSnakeCaseNamingConvention()
            .Options;

        return new DyrepermenDbContext(opt, new FastHusstandContext(husstandId));
    }

    public Task DisposeAsync() => _container.DisposeAsync().AsTask();
}

```

`FastHusstandContext` er en testdobbel som returnerer en fast `HusstandId`. Den finnes kun i testprosjektet — produksjonskoden bruker `DbHusstandContext`.

At migrasjonene kjøres i `InitializeAsync` er selv en test: feiler en migrasjon, feiler hele suiten umiddelbart i stedet for ved neste utrulling.

17.3 Testene som må finnes

Filterprøven — fanger entiteter lagt til uten query-filter:

```

[Fact]
public void Alle_husstandsbundne_entiteter_har_query_filter()
{
    var utenFilter = _db.Model.GetEntityTypes()
        .Where(t => t.ClrType.IsAssignableTo(typeof(IHusstandsbundet)))
        .Where(t => t.GetQueryFilter() is null)
        .Select(t => t.ClrType.Name)
        .ToList();

    Assert.True(utenFilter.Count == 0,
        $"Mangler query filter: {string.Join(", ", utenFilter)}");
}

```

Isolasjonsprøven — den viktigste testen i prosjektet:

```
[Fact]
public async Task Husstand_ser_ikke_annen_husstands_dyr()
{
    await using var a = _fixture.LagContext(husstandId: 1);
    a.Dyr.Add(new Dyr { HusstandId = 1, Navn = "Luna", Art = Art.Hund });
    await a.SaveChangesAsync();

    await using var b = _fixture.LagContext(husstandId: 2);
    Assert.Empty(await b.Dyr.ToListAsync());
    Assert.Null(await b.Dyr.FirstOrDefaultAsync(d => d.Navn == "Luna"));
}
```

Skriv denne først, før noen funksjon er bygget.

Øvrige integrasjonstester som må dekkes:

Test	Verifiserer
Duplikat chipnummer avvises	<code>ux_dyr_chip</code> og oversettelse av SQLSTATE 23505
To dyr uten chipnummer kan lagres	At tom streng normaliseres til <code>NULL</code>
Chipnummer på deaktivert dyr blokkerer fortsatt	At indeksen ser rader query-filteret skjuler
Fôrplan med både prosent og gram avvises	<code>ck_forplan_verdi</code>
To aktive fôrplaner på samme dyr avvises	<code>ux_forplan_aktiv</code>
Sletting av bruker beholder vektrader	<code>ON DELETE SET NULL</code>
Sletting av dyr fjerner vekt og fôringer	<code>ON DELETE CASCADE</code>
Samtidig redigering gir konflikt	<code>UseXminAsConcurrencyToken</code>

Enhetstester: førberegning ved kjent vekt og prosentsats, måltidsfordeling der resten legges på første måltid og summen er eksakt, kilo/gram-konvertering begge veier med avrunding, og `TomTilNull` for tom streng, mellomrom og gyldig verdi.

17.4 Krav til testene i CI

- Testene kjører på hver push og hver pull request (kapittel 14.5)
- Rødt bygg blokkerer utrulling — grenbeskyttelse på `main` med `Bygg` og `test` som påkrevd sjekk
- Testene skriver ikke til delt tilstand. Hver testklasse får egen container eller egen husstands-ID
- Ingen test avhenger av rekkefølge eller av data en annen test la inn
- Ingen test krever nettverkstilgang utover Docker-registeret

18. Åpne spørsmål

1. **Flere brytere senere?** — mønsteret med `dyr.<funksjon>_aktiv` pluss `husstand_innstilling.<funksjon>_standard` skalerer til nye funksjoner, men to kolonner per funksjon blir uryddig etter fem-seks av dem. Vurder en `dyr_funksjon(dyr_id, funksjon char(1), aktiv boolean)`-tabell hvis antallet vokser.
2. **Historikk på fôrplan** — `ux_forplan_aktiv` tillater kun én aktiv plan per dyr, men gamle planer beholdes med `aktiv = false`. Skal endringshistorikken vises i grensesnittet, eller er den bare revisjonsspor?
3. **Varselkanal** — e-post er enklest å starte med. Push krever PWA eller mobilapp, og bør utsettes til det faktisk savnes.
4. **Utgiftsrapport** — `vetbesok.kostnad_kr` gjør en årsoversikt over dyrekostnader triviell å legge til senere. Verdt å vurdere som fase 8.